

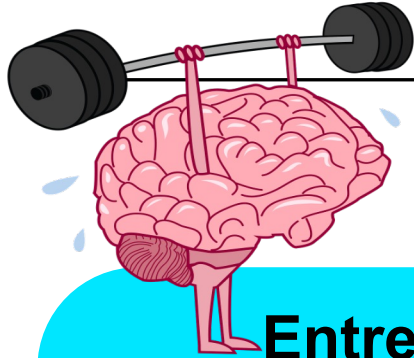
TDVI: Inteligencia Artificial

Forward y Backward propagation

Licenciatura en Tecnología Digital



Automatizando tareas



Entrenamiento

- Objetivo: Ajustar los parámetros del modelo (weights/biases) para *aprender* de los datos
- Herramientas: **Forward & backward propagation** combinado con técnicas de optimización.
- Resultado: Modelo entrenado que puede predecir datos nuevos.



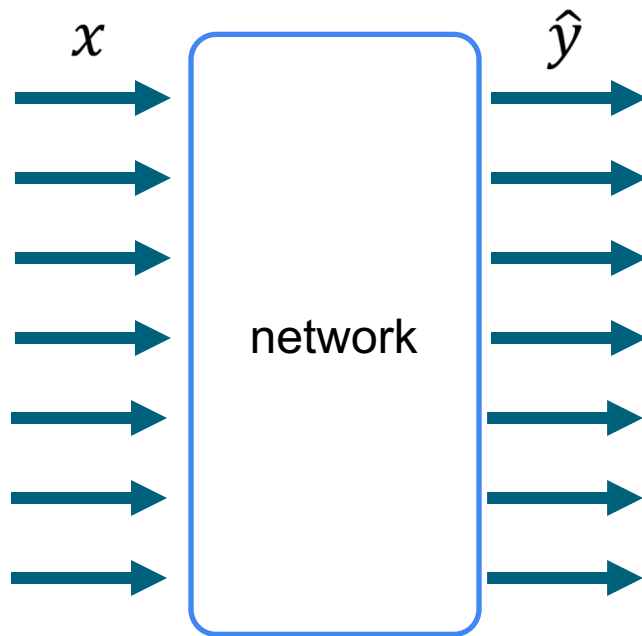
Inferencia

- Objetivo: Usar un modelo entrenado para realizar predicciones (ejem.: clasificación, regresión).
- Herramientas: **Forward propagation** únicamente. Utiliza los pesos y bias aprendido.
- Resultado: Predicción en base a la entrada.

Entrenamiento en dos pasos

● Forward Propagation

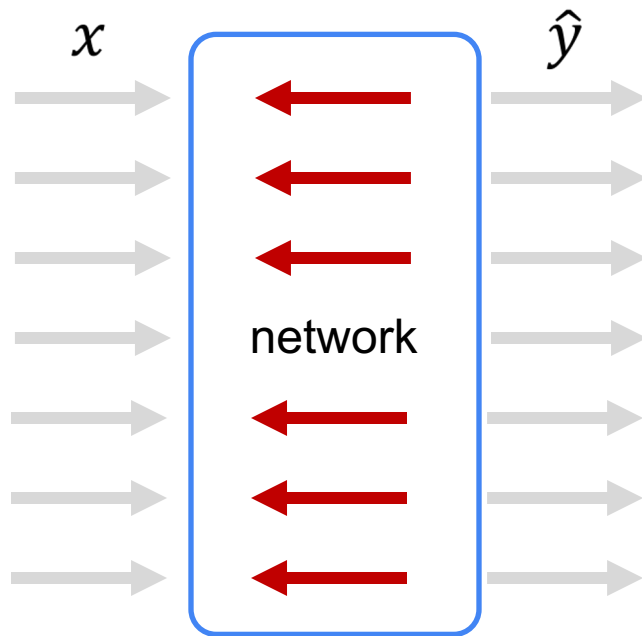
- **Flujo de datos:** Input layer $\rightarrow$ Hidden layers $\rightarrow$ Output layer.
- **Proceso:** Computa el output para un input utilizando los weights y biases
- **Propósito:** Predecir un output (calcularlo)
 - denotado como y_{pred} o $\hat{y}$



Entrenamiento en dos pasos

● Backward Propagation

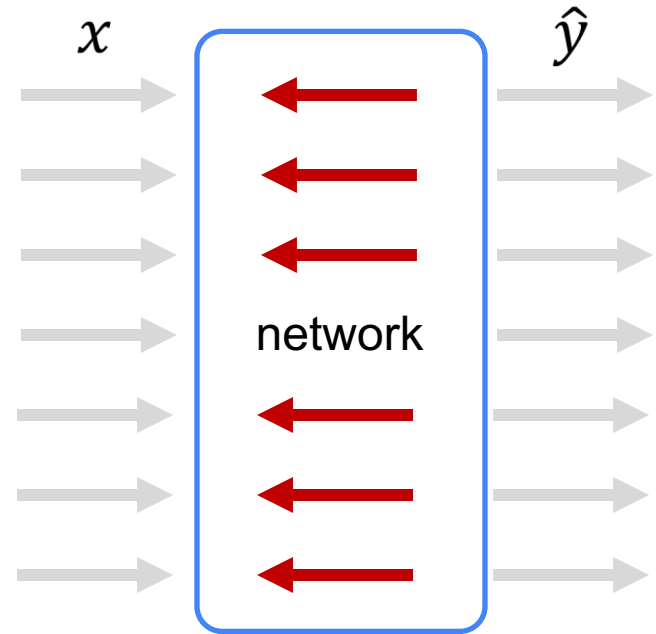
- **Flujo de datos:** Output layer $\rightarrow$ Hidden layers $\rightarrow$ Input layer.
- **Proceso:** Calcular los gradientes (derivadas parciales) de la función “loss” con respecto a los weights y biases utilizando la regla de la cadena.
- **Objetivo:** Computar y actualizar los parámetros del modelo para minimizar el error con respecto de los datos de entrenamiento.



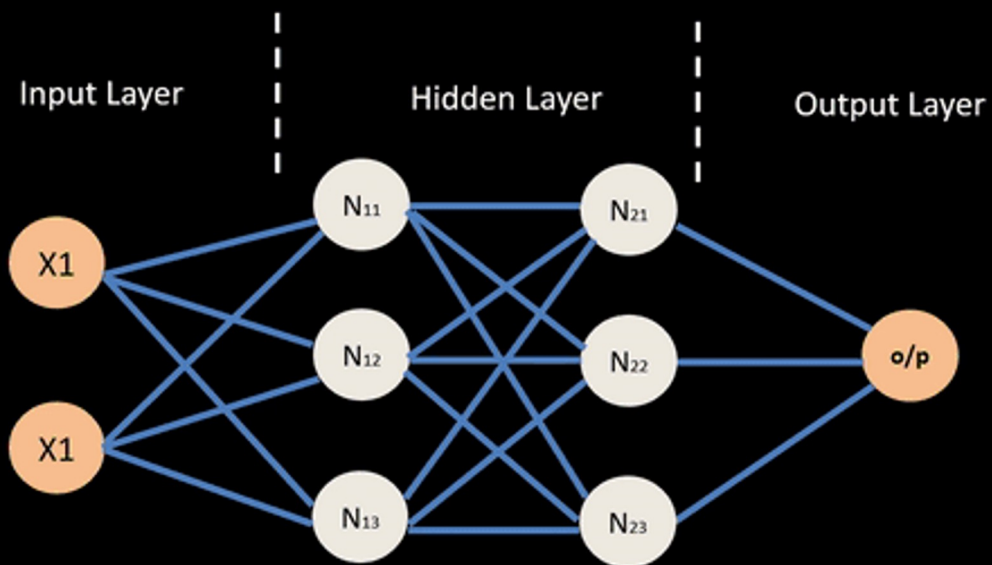
Entrenamiento en dos pasos

- **Flexibilidad de entrenamiento entre capas**

- **Capas entrenables:** Capas cuyos weights & biases son ajustados durante backpropagation.
- **Capas no entrenables:** Capas fijadas durante el entrenamiento, en general para mantener features (características) ya aprendidas previamente.



Neural Network – Backpropagation



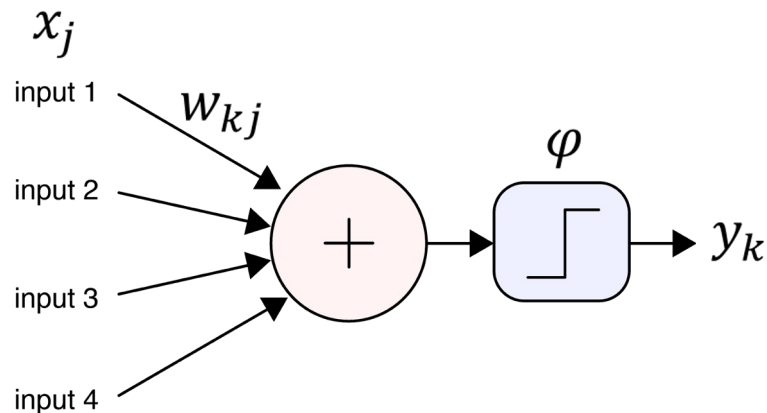
Agenda

1. Introducción
2. **Forward Propagation/Pass**
3. Vector computation
4. Backward Propagation/Pass
 - a. Ejemplo
5. Grafos computacionales
6. Conjunto de observaciones
7. Resumen



Redes neuronales artificiales

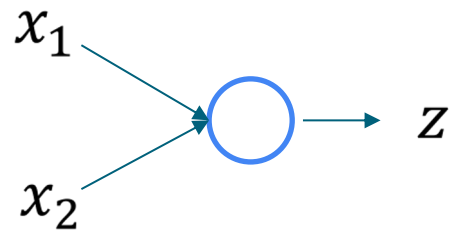
- Generalmente se dibuja sin los pesos ni bias
- Formalmente por una unidad k :
 - Inputs: x_j
 - Weights: w_{kj}
 - Bias: w_{k0} ($m+1$ inputs, $x_0 = 1$)
 - Funcion de activacion: φ
 - Output: y_k



$$y_k = \varphi \left(\sum_{j=0}^m w_{kj} x_j \right)$$

Forward Pass: ejemplo con un nodo

- **Ejemplo:** 2 inputs y 1 unidad.



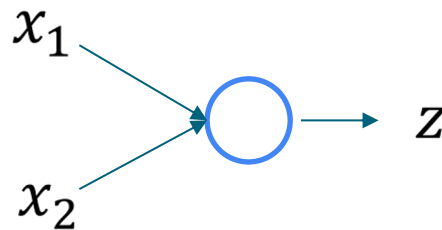
Forward Pass: ejemplo con un nodo

- **Ejemplo:** 2 inputs y 1 unidad.

- Combinación lineal:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + w_0 \cdot 1$$

- x_1, x_2 son input
- w_1, w_2 son pesos (weights)
- $w_0 \cdot 1$ es el término bias
- z es la combinación lineal para la unidad (previo a la función de activación)



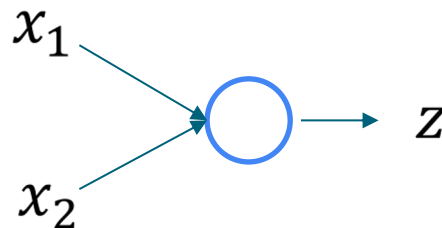
Forward Pass: ejemplo con un nodo

- **Ejemplo:** 2 inputs y 1 unidad.

- Combinación lineal:

$$Z = w_1 \cdot x_1 + w_2 \cdot x_2 + w_0 \cdot b$$

- x_1, x_2 son input
- w_1, w_2 son pesos (weights)
- **b es el término bias**
- z es la combinación lineal para la unidad (previo a la función de activación)



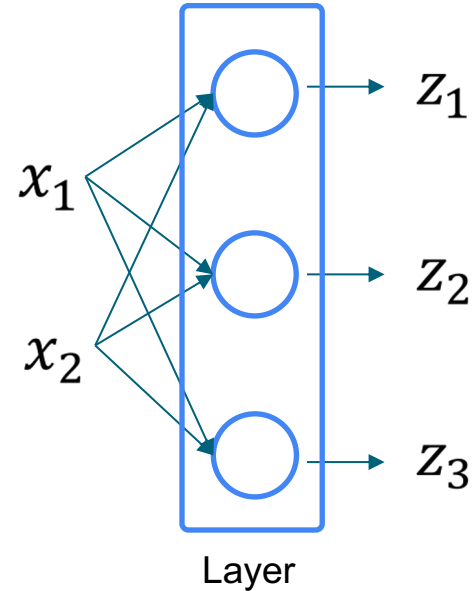
Forward Pass: ejemplo con un nodo

- **Ejemplo:** 2 inputs y 3 unidades.
- Combinación lineal:

$$Z_1 = w_{11} \cdot x_1 + w_{21} \cdot x_2 + b_1$$

$$Z_2 = w_{12} \cdot x_1 + w_{22} \cdot x_2 + b_2$$

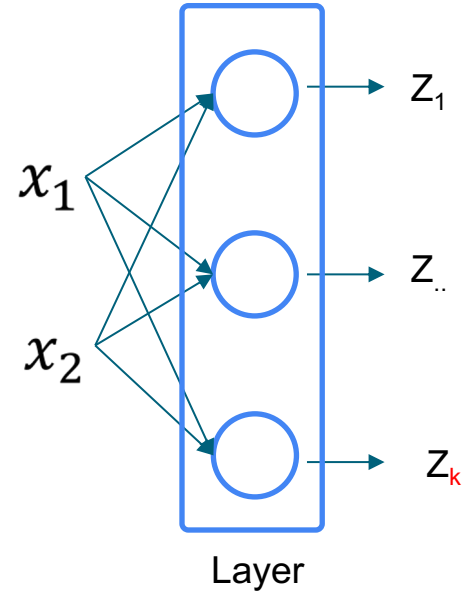
$$Z_3 = w_{13} \cdot x_1 + w_{23} \cdot x_2 + b_3$$



Forward Pass: ejemplo con un nodo

- **Ejemplo:** 2 inputs y **k** unidades.
- Combinación lineal:

$$z_{\mathbf{k}} = w_{1\mathbf{k}} \cdot x_1 + w_{2\mathbf{k}} \cdot x_2 + \mathbf{b}_{\mathbf{k}}$$



Forward Pass: ejemplo con un nodo

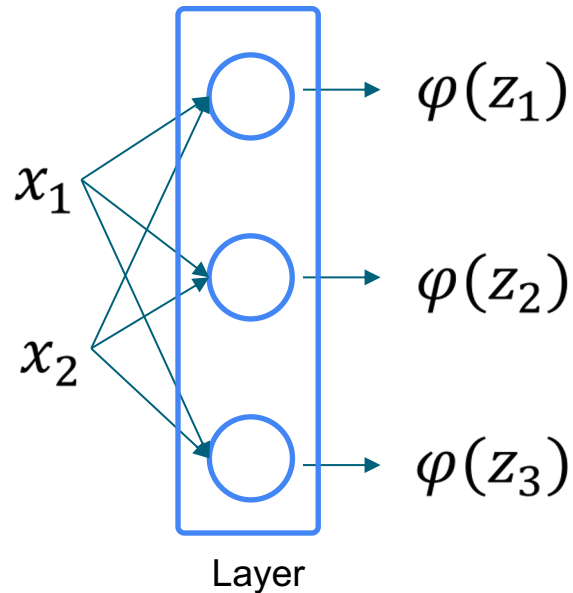
- **Ejemplo:** 2 inputs y **k** unidades.

- Combinación lineal:

$$z_k = w_{1k} \cdot x_1 + w_{2k} \cdot x_2 + b_k$$

- Función de activación

$$a_k = \varphi(z_k)$$



Forward Pass: ejemplo con un nodo

- **Ejemplo:** 2 inputs y **k** unidades.

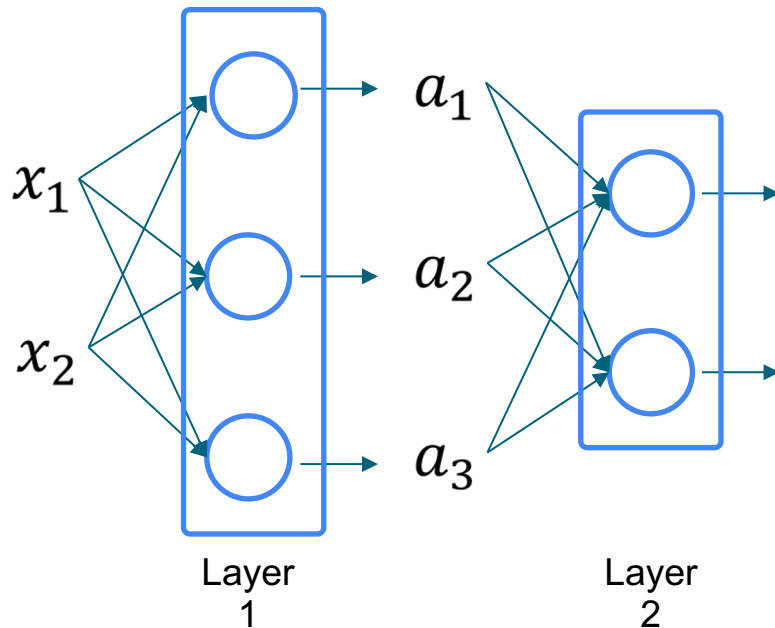
- Combinación lineal:

$$z_k = w_{1k} \cdot x_1 + w_{2k} \cdot x_2 + b_k$$

- Función de activación

$$a_k = \varphi(z_k)$$

- La próxima capa toma a_k como input



Forward Pass: ejemplo con un nodo

- **Ejemplo:** 2 inputs y **k** unidades.

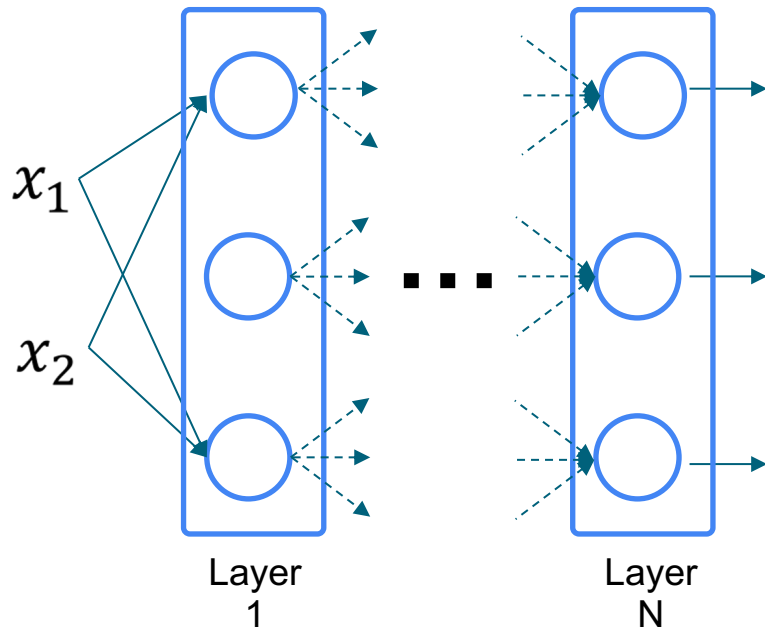
- Combinación lineal:

$$z_k = w_{1k} \cdot x_1 + w_{2k} \cdot x_2 + b_k$$

- Función de activación

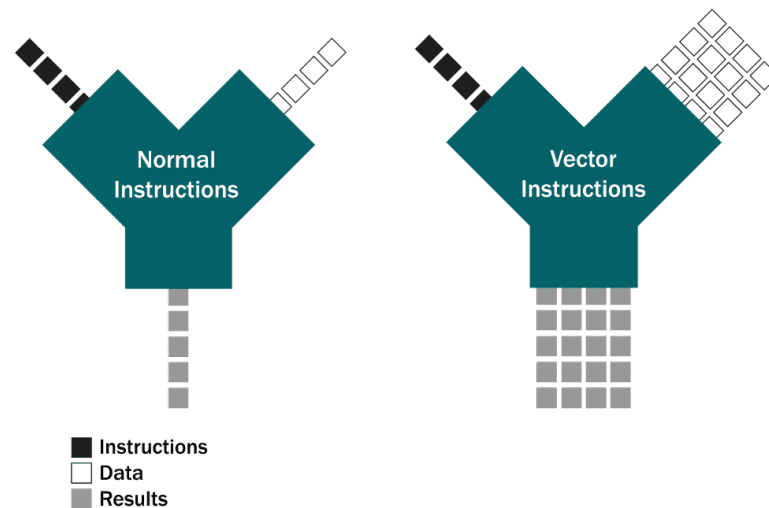
$$a_k = \varphi(z_k)$$

- Cada capa toma a **la capa anterior** como input



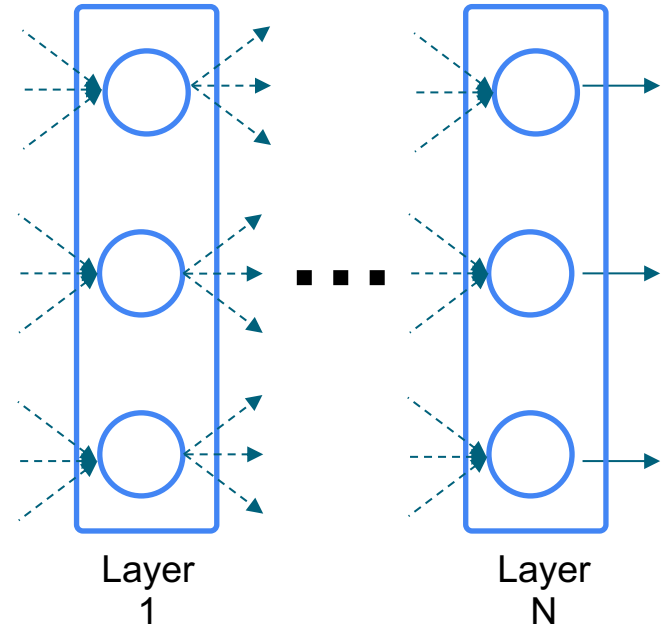
Agenda

1. Introducción
2. Forward Propagation/Pass
3. **Vectorización**
4. Backward Propagation/Pass
 - a. Ejemplo
5. Grafos computacionales
6. Conjunto de observaciones
7. Resumen



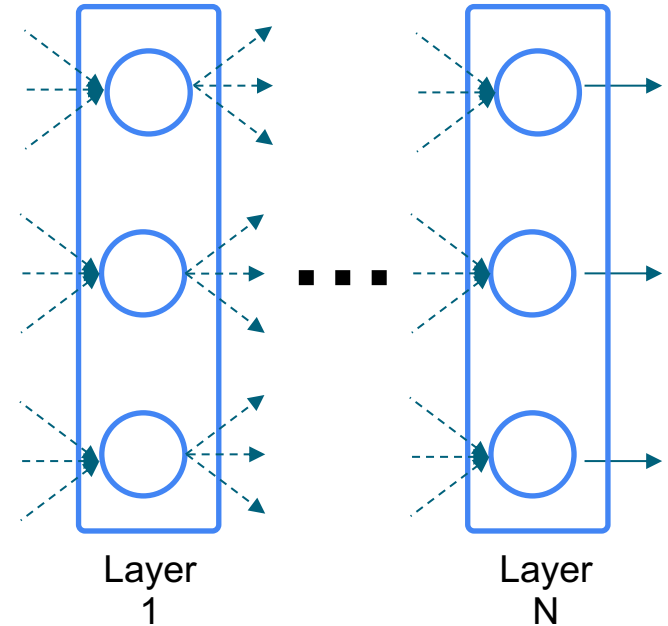
Cálculos entre capas FC

- Cada unidad está conectada a todas las unidades de la capa anterior.



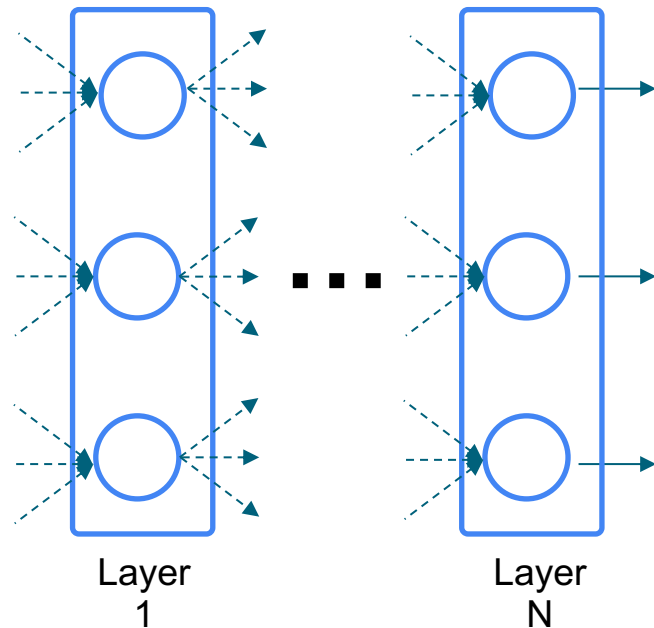
Cálculos entre capas FC

- Cada unidad está conectada a todas las unidades de la capa anterior.
- Transforma el input utilizando los pesos y bias



Cálculos entre capas FC

- Cada unidad está conectada a todas las unidades de la capa anterior.
- Transforma el input utilizando los pesos y bias
- **La vectorización** simplifica las operaciones y permite la paralelización y aceleración por hardware.



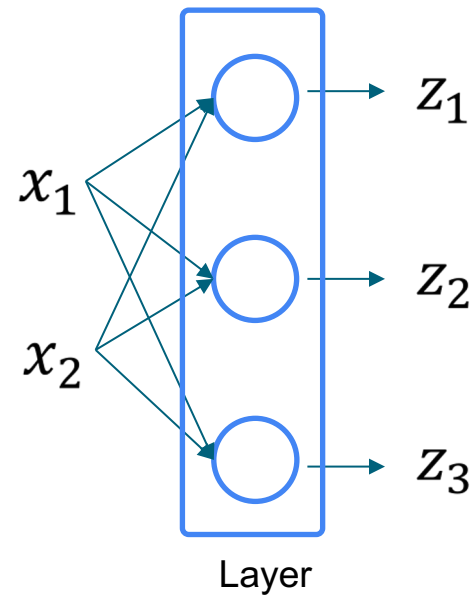
Cálculos entre capas FC

- Matriz de pesos (W): almacena los pesos para las conexiones entre capas.
- Vector input (x): almacena los valores de entrada.
- Vector bias (b): Almacena los biases de cada neurona.
- Combinación lineal:

$$z = W^T x + b$$

Forward Pass: Ejemplo FC

- **Ejemplo:** 2 inputs y 3 unidades.

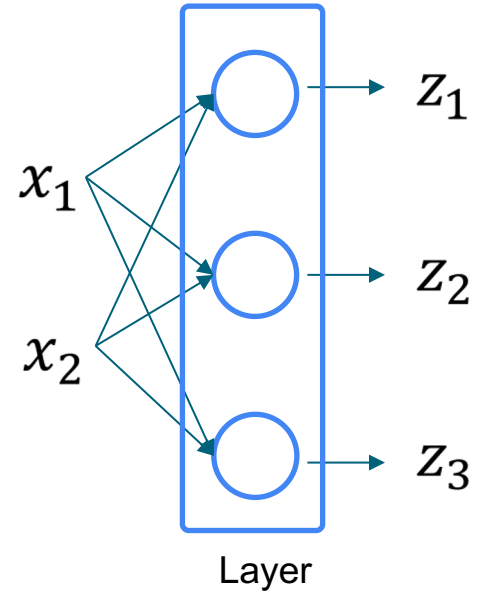


Forward Pass: Ejemplo FC

- **Ejemplo:** 2 inputs y 3 unidades.

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

#input × #units

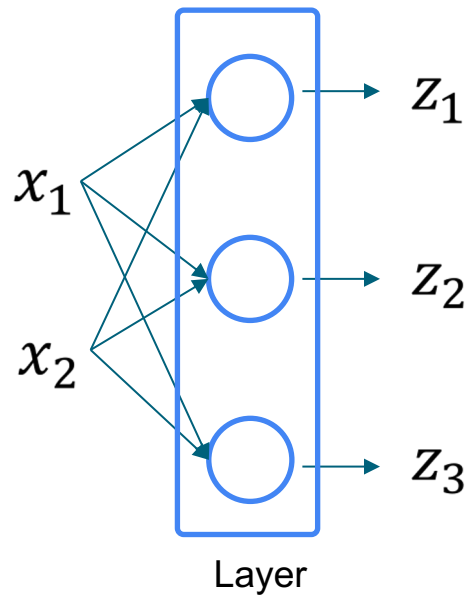


Forward Pass: Ejemplo FC

- **Ejemplo:** 2 inputs y 3 unidades.

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

$$\mathbf{z} = \begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

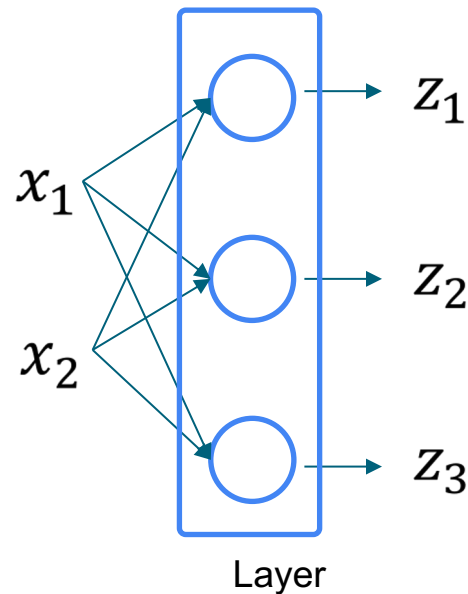


Forward Pass: Ejemplo FC

- **Ejemplo:** 2 inputs y 3 unidades.

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

$$\mathbf{z} = \begin{bmatrix} w_{11}x_1 + w_{21}x_2 + b_1 \\ w_{12}x_1 + w_{22}x_2 + b_2 \\ w_{13}x_1 + w_{23}x_2 + b_3 \end{bmatrix}$$



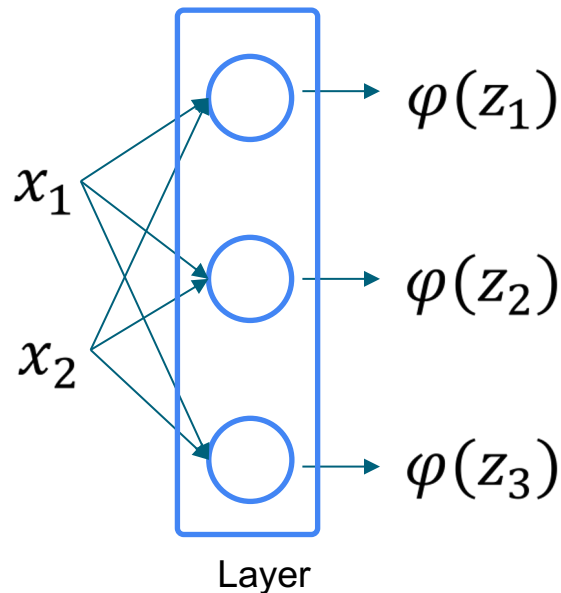
Forward Pass: Ejemplo FC

- **Ejemplo:** 2 inputs y 3 unidades.

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

- Función de activación por elemento:

$$\mathbf{a} = \begin{bmatrix} \varphi(w_{11}x_1 + w_{21}x_2 + b_1) \\ \varphi(w_{12}x_1 + w_{22}x_2 + b_2) \\ \varphi(w_{13}x_1 + w_{23}x_2 + b_3) \end{bmatrix}$$

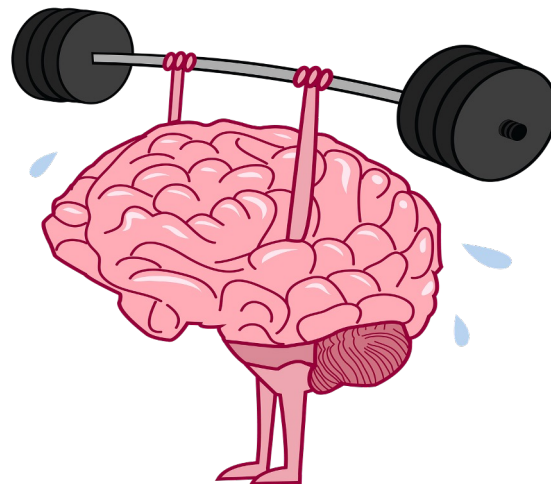


Ventajas de la vectorización

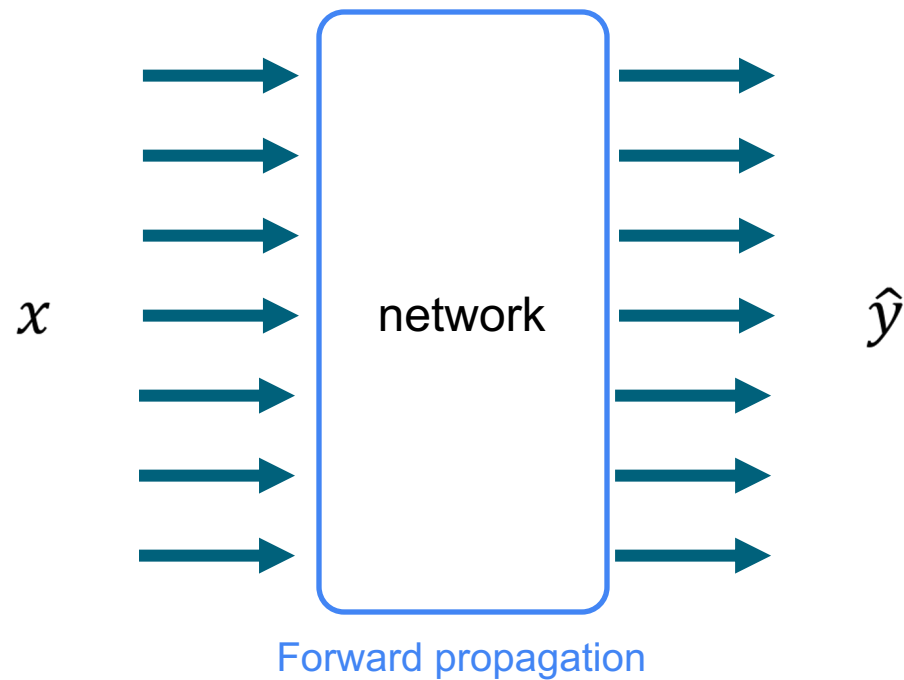
- **Escalabilidad:** Fácil de manejar redes grandes.
- **Claridad:** Simplifica la notación, visualización y comunicación.
- **Performance:** Computos mas rapidos con librerías optimizadas (aceleración por hardware con GPUs)
- **Deep Learning frameworks:** TensorFlow, PyTorch, Keras.

Agenda

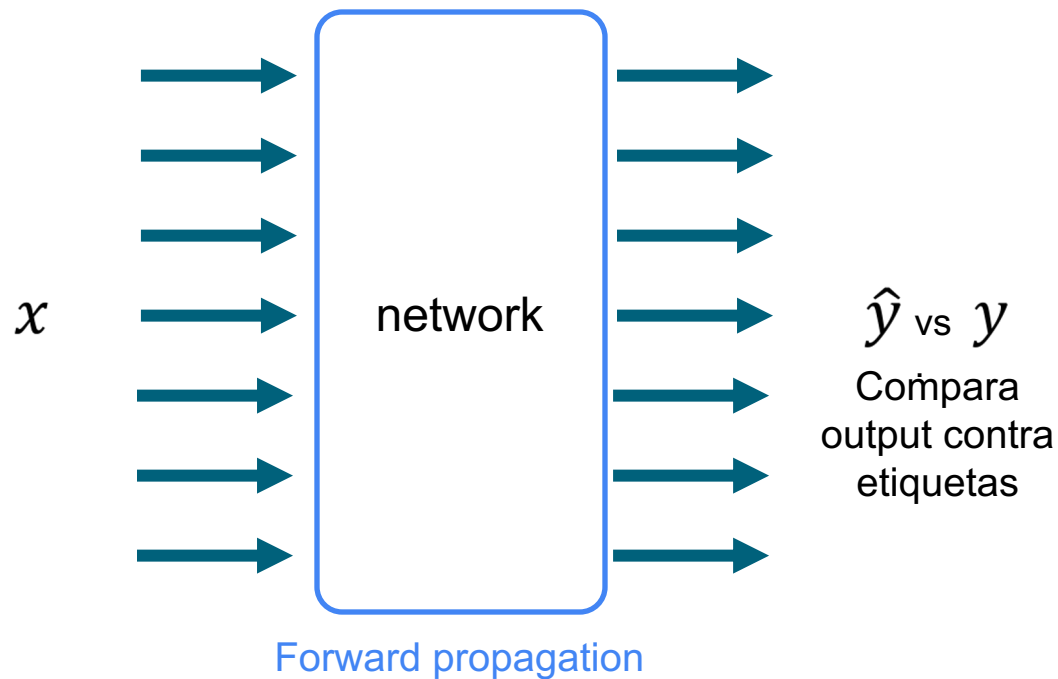
1. Introducción
2. Forward Propagation/Pass
3. Vector computation
4. **Backward Propagation/Pass**
 - a. Ejemplo
5. Grafos computacionales
6. Conjunto de observaciones
7. Resumen



Forward Propagation

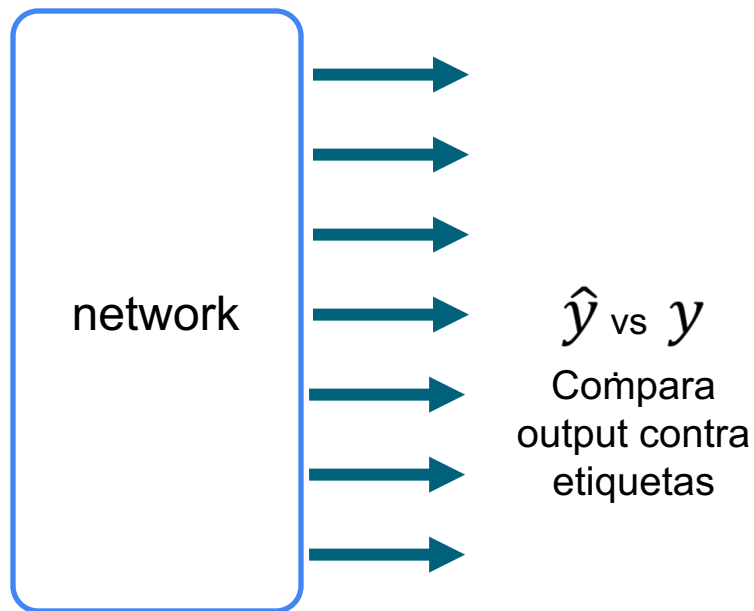


Forward Propagation



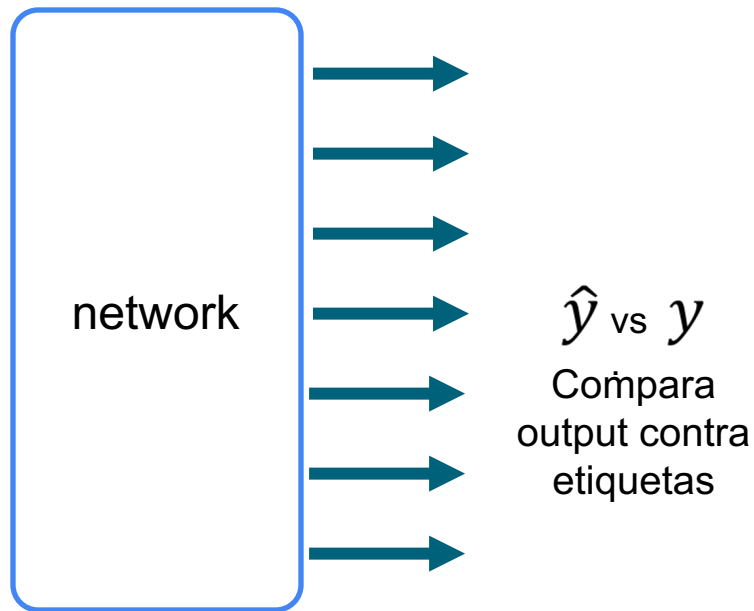
Measuring Prediction Error

- **Función de pérdida (loss):** cuantifica el error entre las predicciones de la red y las etiquetas.
- Guía el entrenamiento dando feedback.



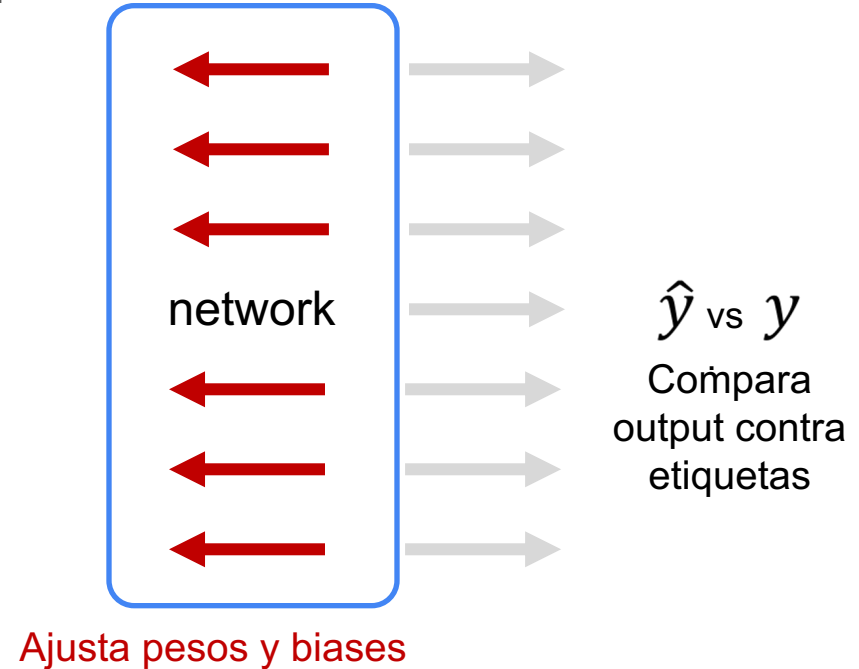
Measuring Prediction Error

- **Función de pérdida (loss):** cuantifica el error entre las predicciones de la red y las etiquetas.
- Guía el entrenamiento dando feedback.
- **Ejemplos:**
 - Mean Squared Error (regression)
 - Cross-Entropy (classification).



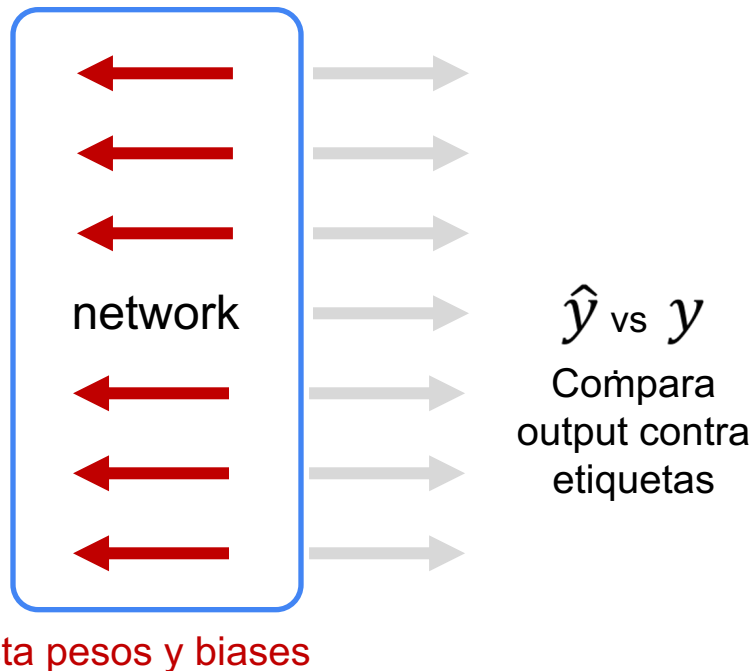
What is Backward Propagation?

- **Objetivo del entrenamiento:** minimizar el valor de la función de pérdida actualizando los pesos y biases.



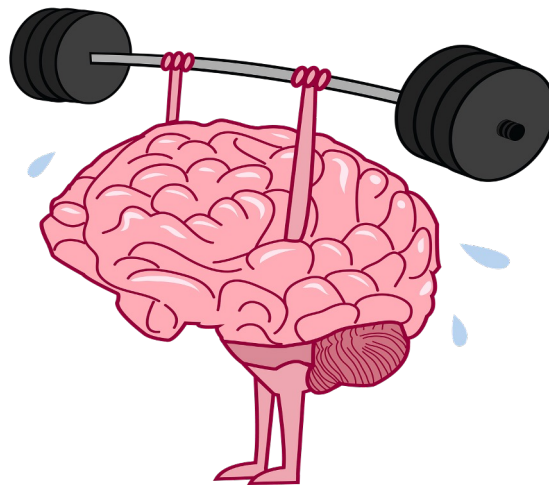
What is Backward Propagation?

- **Objetivo del entrenamiento:** minimizar el valor de la función de pérdida actualizando los pesos y biases.
- **Backpropagation** es el algoritmo que computa los cambios en los parámetros del modelo para reducir la diferencia entre el valor predicho y la etiqueta.



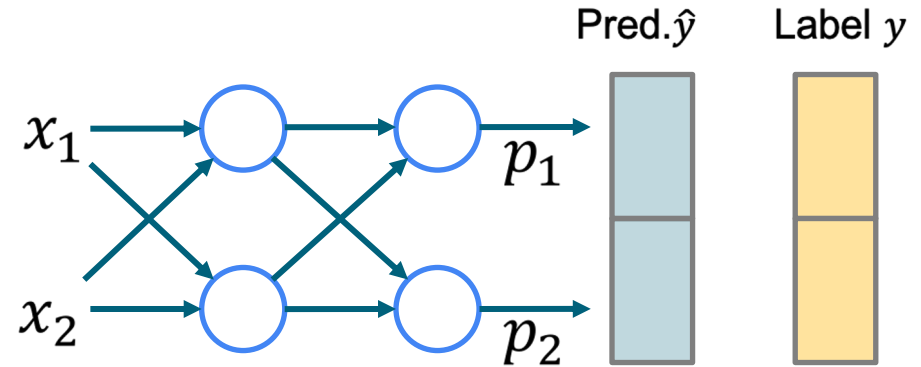
Agenda

1. Introducción
2. Forward Propagation/Pass
3. Vector computation
4. **Backward Propagation/Pass**
 - a. Ejemplo



Ejemplo de Back propagation

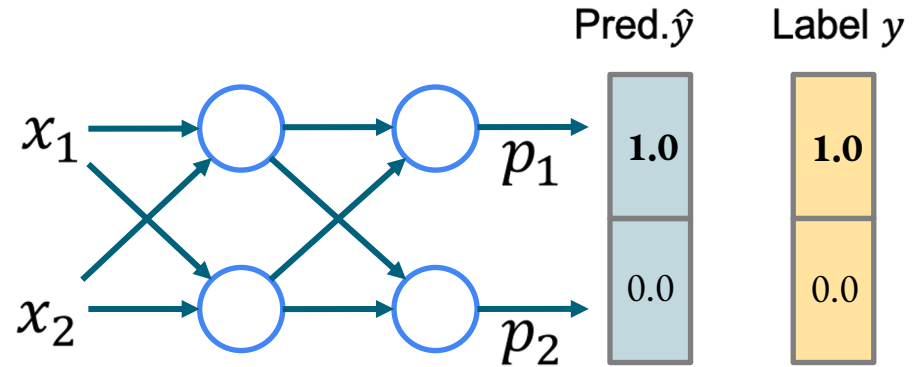
Clasificador de 2 clases: A y B.



Ejemplo de Back propagation

Clasificador de 2 clases: A y B.

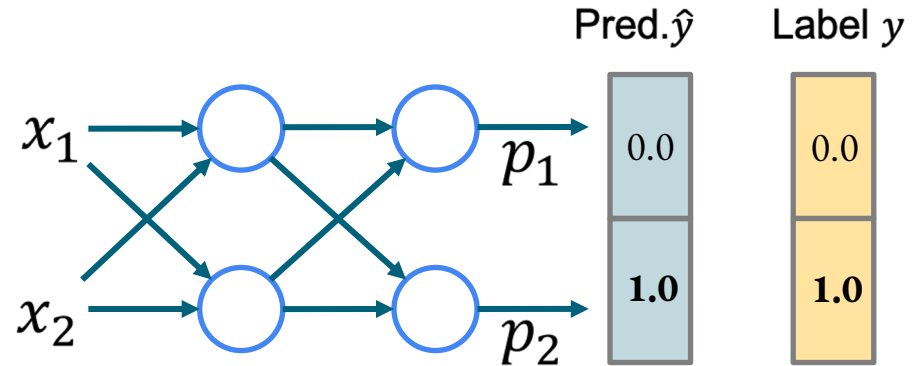
- Ideal: una muestra que pertenece a:
 - la clase A tiene que tener un output $p_1=1.0$ y $p_2=0.0$



Ejemplo de Back propagation

Clasificador de 2 clases: A y B.

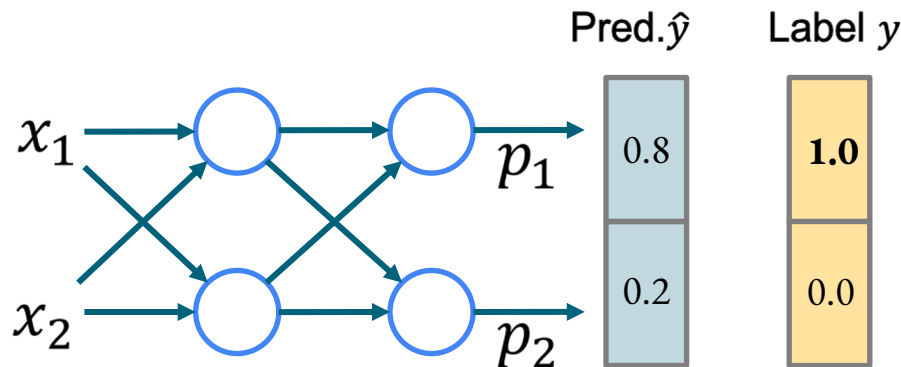
- Ideal: una muestra que pertenece a:
 - la clase A tiene que tener un output $p_1=1.0$ y $p_2=0.0$
 - la clase B tiene que tener un output $p_1=0.0$ y $p_2=1.0$



Ejemplo de Back propagation

Clasificador de 2 clases: A y B.

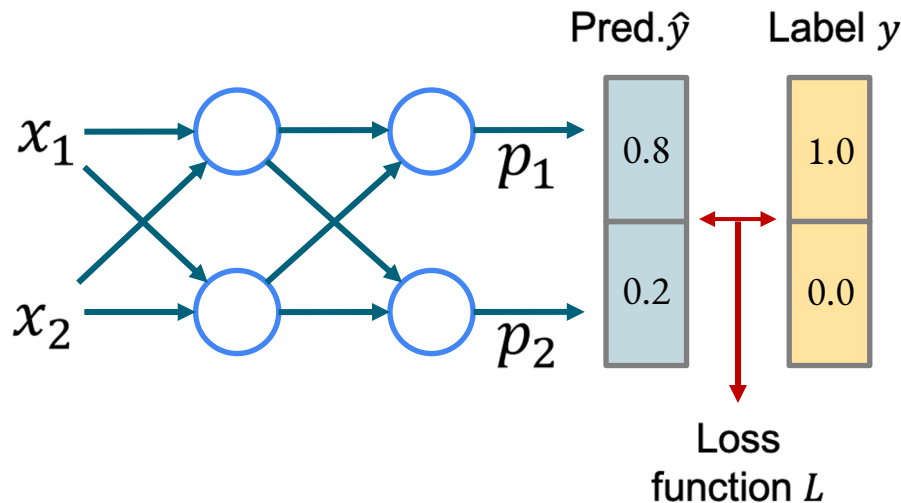
- Ideal: una muestra que pertenece a:
 - la clase A tiene que tener un output $p_1=1.0$ y $p_2=0.0$
 - la clase B tiene que tener un output $p_1=0.0$ y $p_2=1.0$
- **Realidad:** las redes neuronales no son tan certeras!



Ejemplo de Back propagation

Clasificador de 2 clases: A y B.

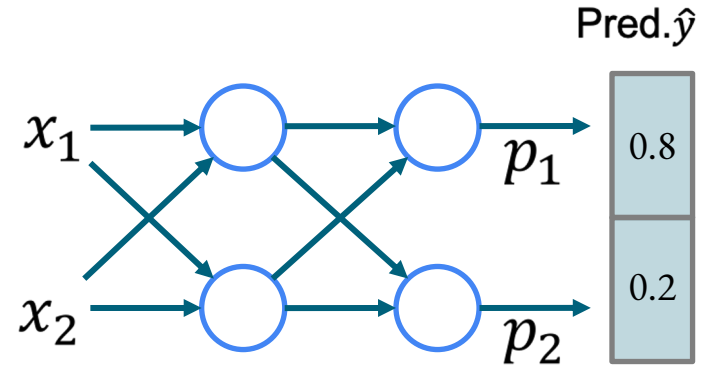
- Ideal: una muestra que pertenece a:
 - la clase A tiene que tener un output $p_1=1.0$ y $p_2=0.0$
 - la clase B tiene que tener un output $p_1=0.0$ y $p_2=1.0$
- **Realidad:** las redes neuronales no son tan certeras!
 - La función de pérdida cuantifica la diferencia.



Ejemplo de Back propagation

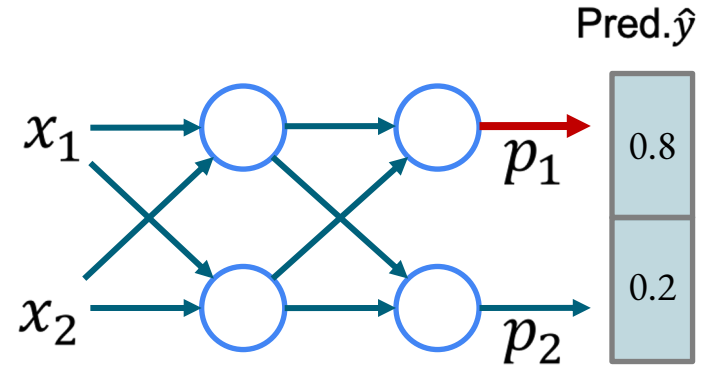
- Objetivo:

- p_1 cerca de 1.0
- p_2 cerca de 0.0



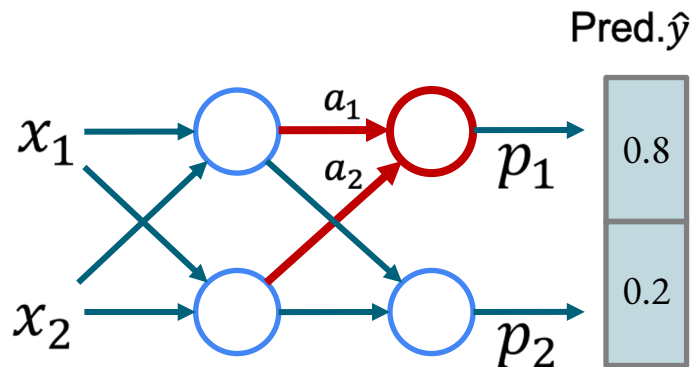
Ejemplo de Back propagation

- Objetivo:
 - p_1 cerca de 1.0
 - p_2 cerca de 0.0
- No podemos modificar directamente la activación de las capas



Ejemplo de Back propagation

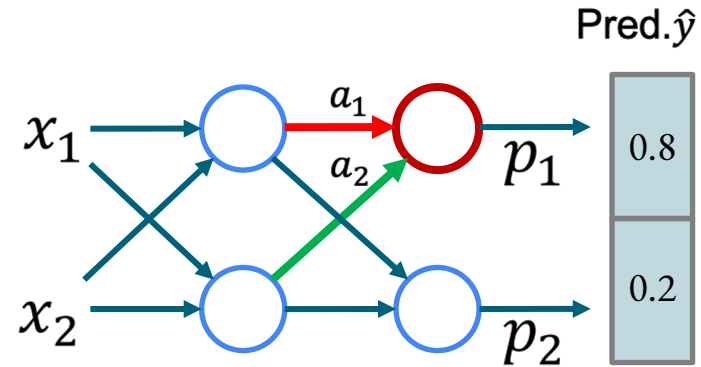
- Objetivo:
 - p_1 cerca de 1.0
 - p_2 cerca de 0.0
- No podemos modificar directamente la activación de las capas
 - Modificamos el bias, los pesos o las activaciones de la capa anterior (a_i)



$$0.8 = \varphi(w_1 \times a_1 + w_2 \times a_2 + b)$$

Ejemplo de Back propagation

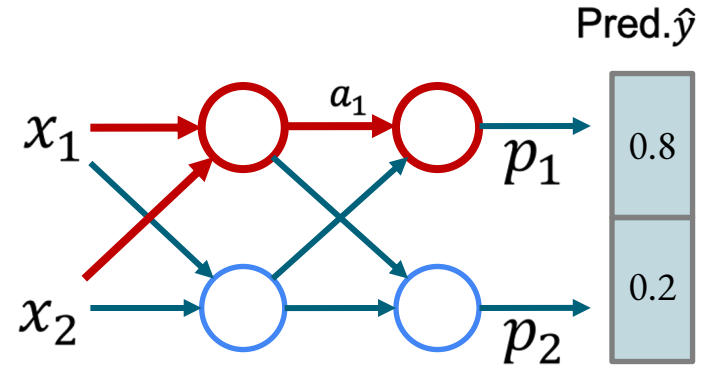
- No todos los pesos contribuyen de la misma manera



$$0.8 = \varphi(w_1 \times a_1 + w_2 \times a_2 + b)$$

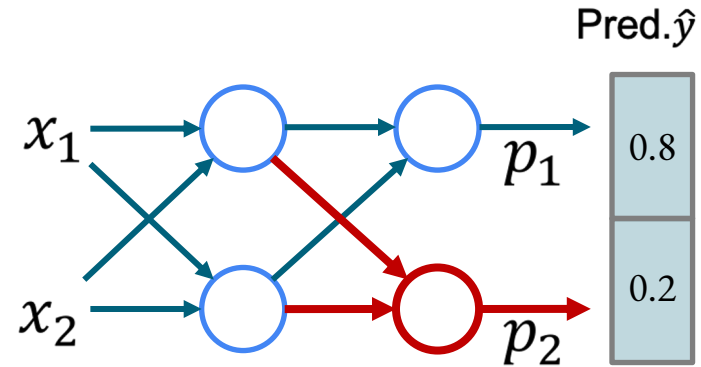
Ejemplo de Back propagation

- No todos los pesos contribuyen de la misma manera
- Las unidades de activación son fuertemente dependientes (para atrás)



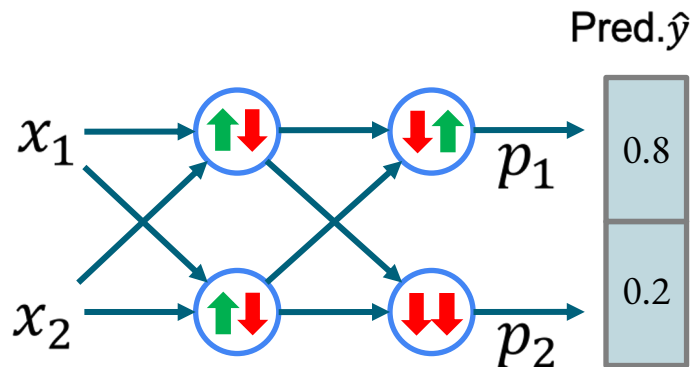
Ejemplo de Back propagation

- No todos los pesos contribuyen de la misma manera
- Las unidades de activación son fuertemente dependientes (para atrás)
- Los demás outputs también son importantes.



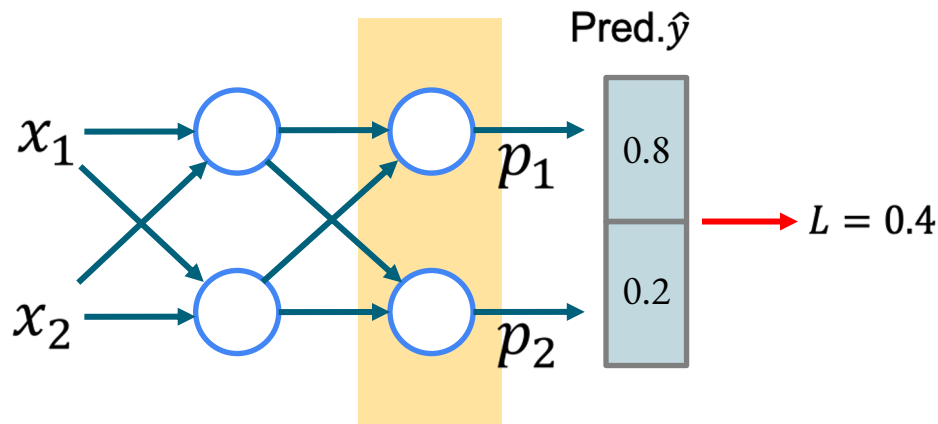
Backward Propagation Example

- Se calcula la derivada de la función de pérdida (loss) con respecto a cada peso en la red
 - Con el algoritmo de backward propagation



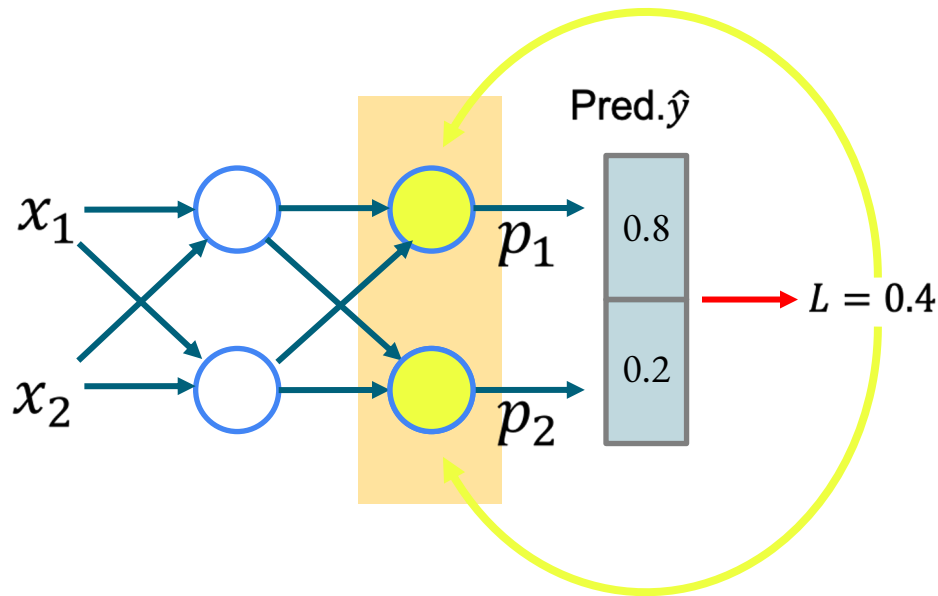
Back propagation: Paso 1

- **Paso 1:** Se computa cuán dependientes son el error en la salida de las neuronas de la última capa.



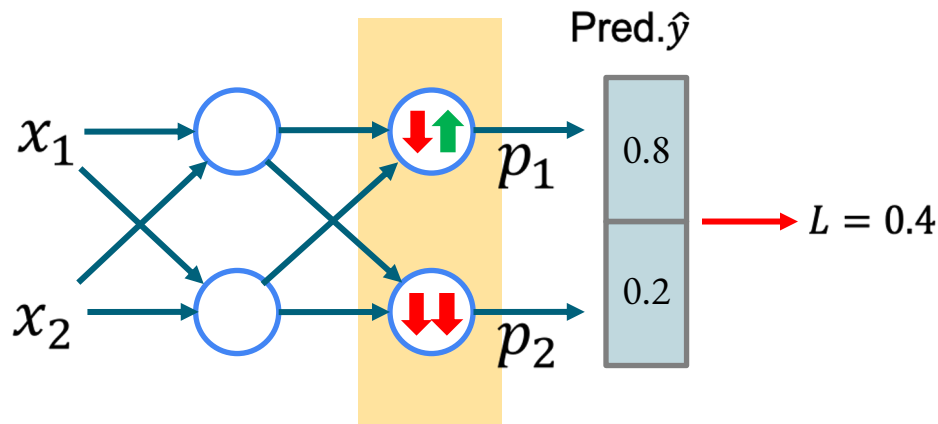
Back propagation: Paso 1

- **Paso 1:** Se computa cuán dependientes son el error en la salida de las neuronas de la última capa.



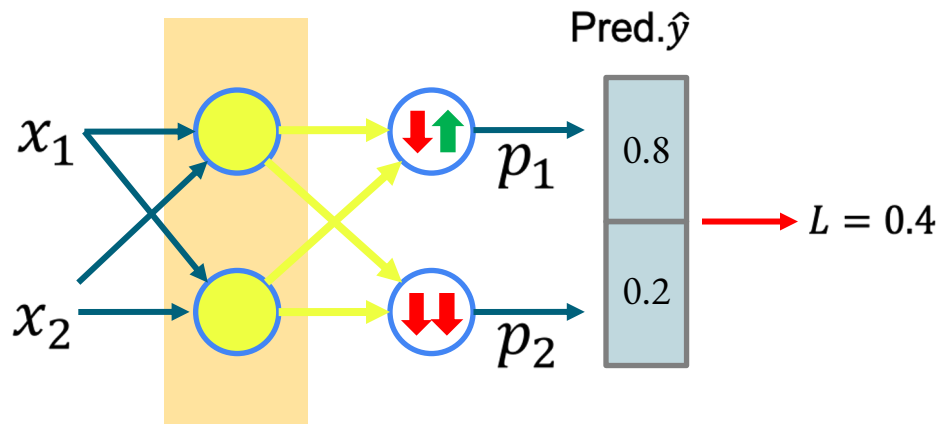
Back propagation: Paso 1

- **Paso 1:** Se computa cuán dependientes son el error en la salida de las neuronas de la última capa.



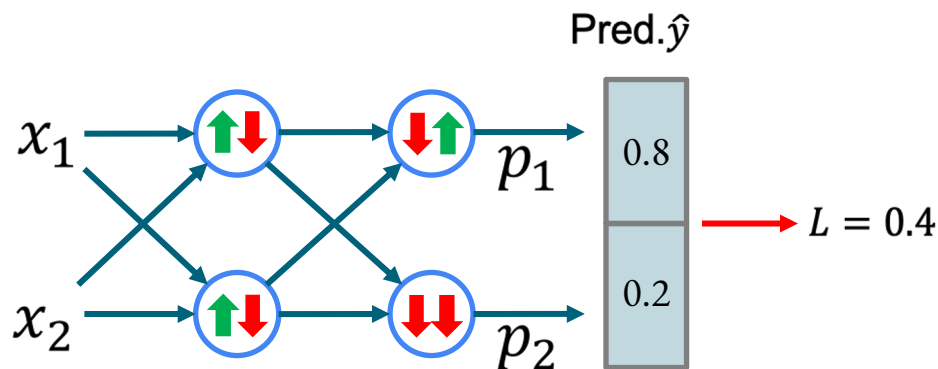
Back propagation: Paso 2

- **Paso 1:** Se computa cuán dependientes son el error en la salida de las neuronas de la última capa.
- **Paso 2:** Vamos para atrás (recursivamente) computando las actualizaciones necesarias en los pesos previos.



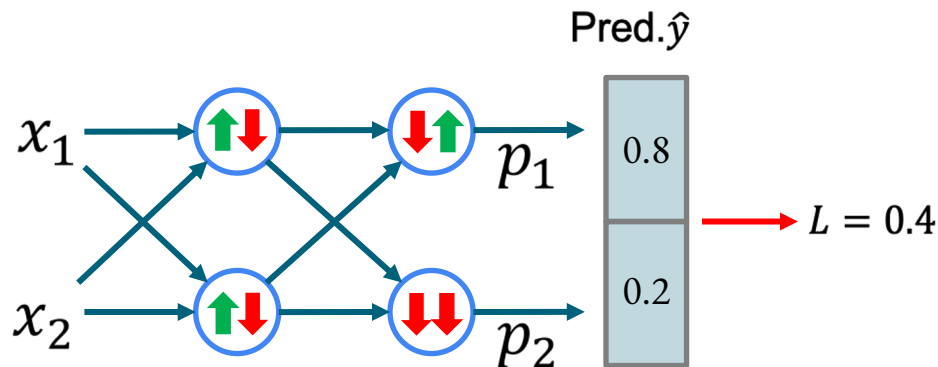
Back propagation: Paso 2

- **Paso 1:** Se computa cuán dependientes son el error en la salida de las neuronas de la última capa.
- **Paso 2:** Vamos para atrás (recursivamente) computando las actualizaciones necesarias en los pesos previos.



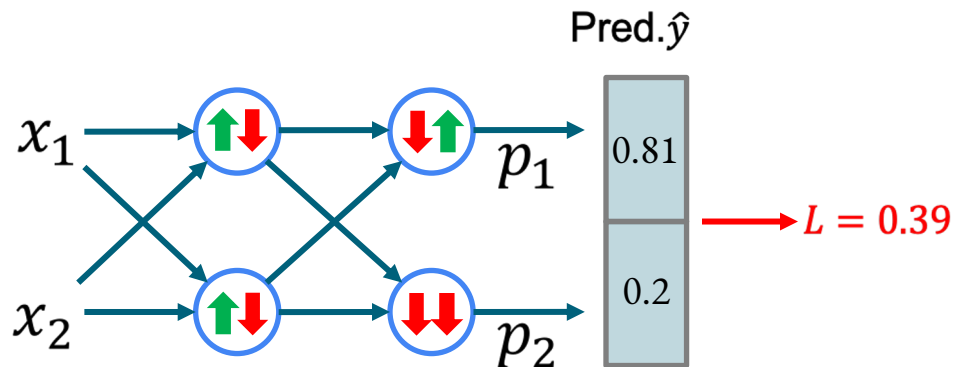
Back propagation: Paso 3

- **Paso 3: Actualizamos todos los pesos** (un poquito) en la dirección indicada.



Back propagation: Paso 3

- **Paso 3: Actualizamos todos los pesos** (un poquito) en la dirección indicada.
- **La función de pérdida decreció** (para las observaciones actuales)



Pros and Cons

- **Eficiencia:** Solo requiere una pasada “forward” y una “backward” para cada observación.
- **Sensibilidad a la arquitectura de la red:** La estabilidad de backpropagation depende de esta, incluyendo profundidad y tipo de conexiones.

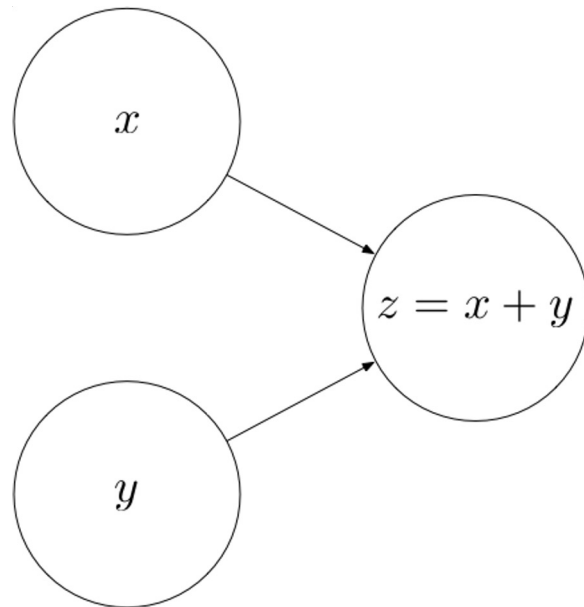
Agenda

1. Introducción
2. Forward Propagation/Pass
3. Vector computation
4. Backward Propagation/Pass
 - a. Ejemplo
5. Grafos computacionales
6. Conjunto de observaciones
7. Resumen

$$(g \circ f)'(x) = g'(f(x)) \cdot f'(x)$$

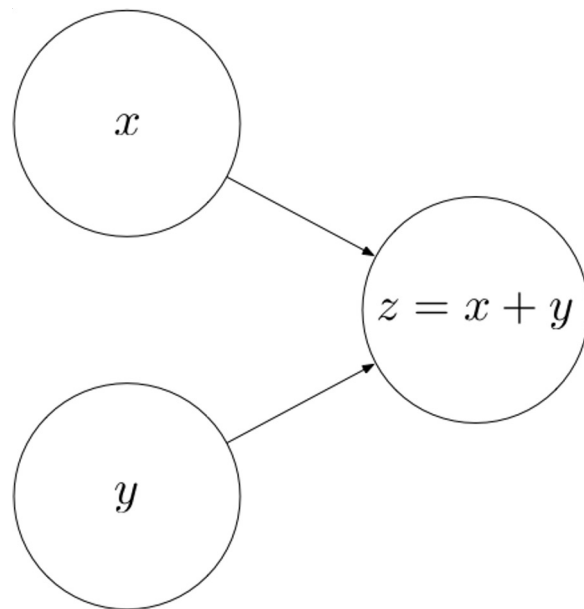
Grafos computacionales

- Backpropagation es simple de entender, pero no tanto de implementar.



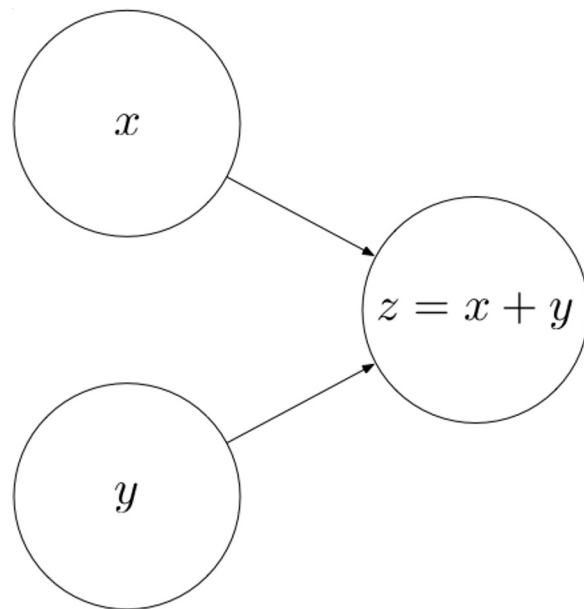
Grafos computacionales

- Backpropagation es simple de entender, pero no tanto de implementar.
- Las librerías de Deep Learning computan automáticamente los gradientes w.r.t weights/pesos.



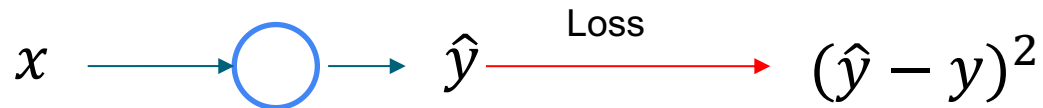
Grafos computacionales

- Backpropagation es simple de entender, pero no tanto de implementar.
- Las librerías de Deep Learning computan automáticamente los gradientes w.r.t weights/pesos.
 - Mantienen trazabilidad de las operaciones de tensores utilizando grafos computacionales.



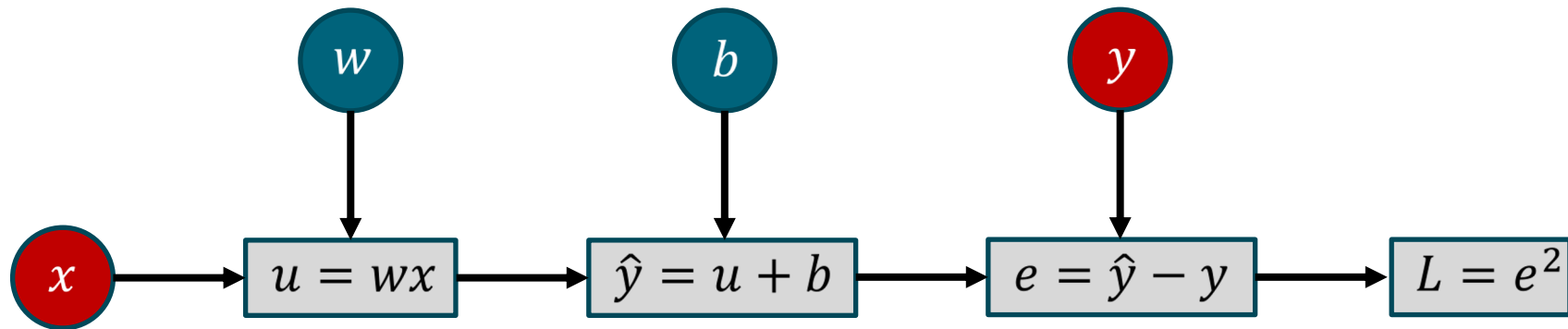
Grafos computacionales

- Ejemplo: $((wx + b) - y)^2$



Grafos computacionales

- Ejemplo: $((wx + b) - y)^2$



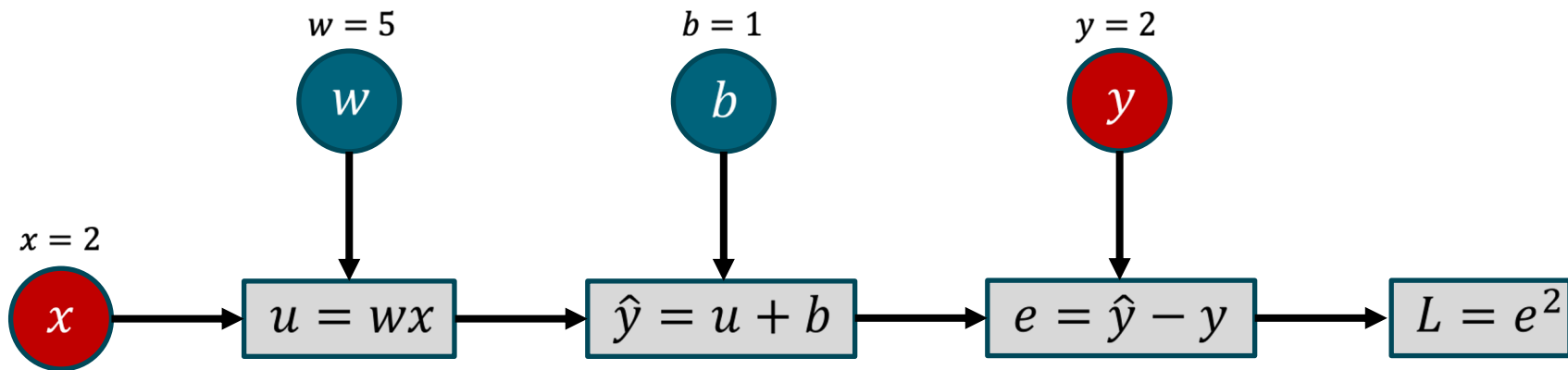
 Parameter
Nodes

 Input
Nodes

 Computation
Nodes

Grafos computacionales: Forward Pass

- Ejemplo: $((wx + b) - y)^2$



Parameter
Nodes



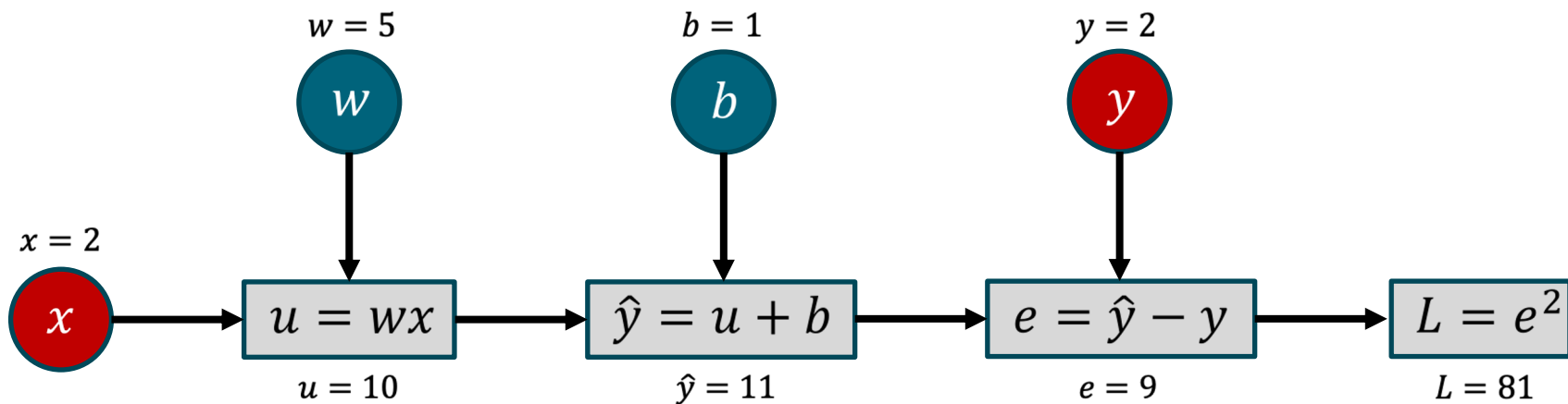
Input
Nodes



Computation
Nodes

Grafos computacionales: Forward Pass

- Ejemplo: $((wx + b) - y)^2$



Parameter
Nodes



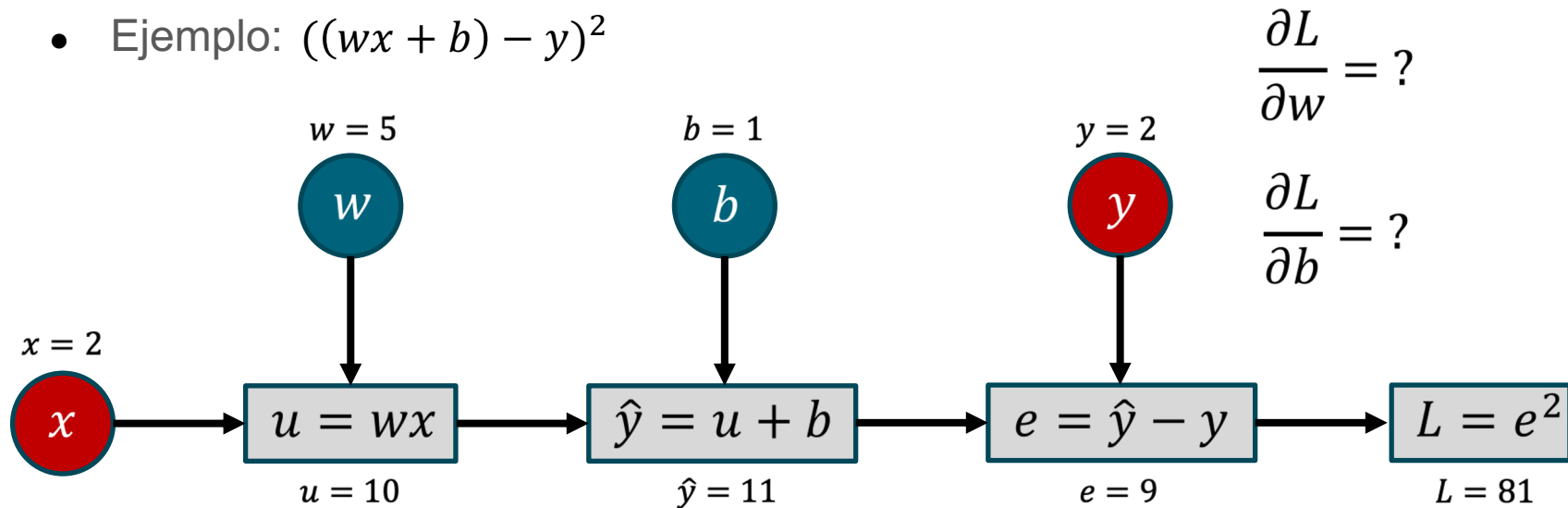
Input
Nodes



Computation
Nodes

Grafos computacionales: Backward Pass

- Ejemplo: $((wx + b) - y)^2$



Parameter
Nodes

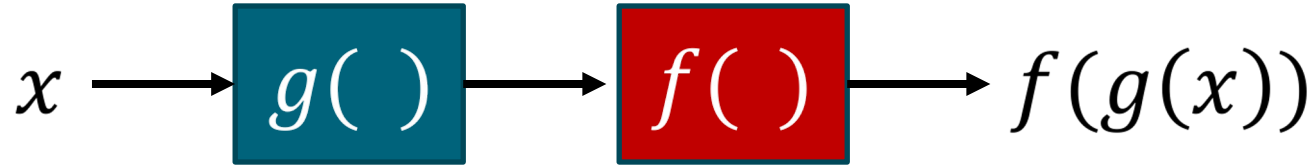


Input
Nodes



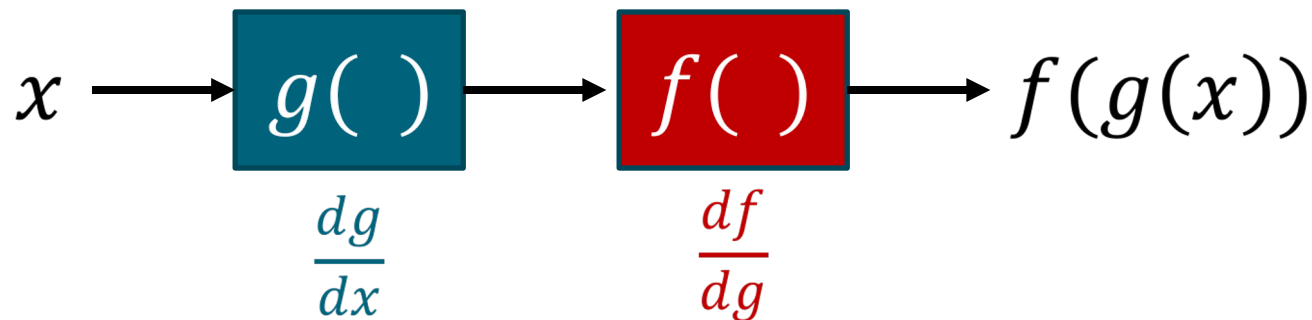
Computation
Nodes

Regla de la cadena



$$\frac{df(g(x))}{dx} =$$

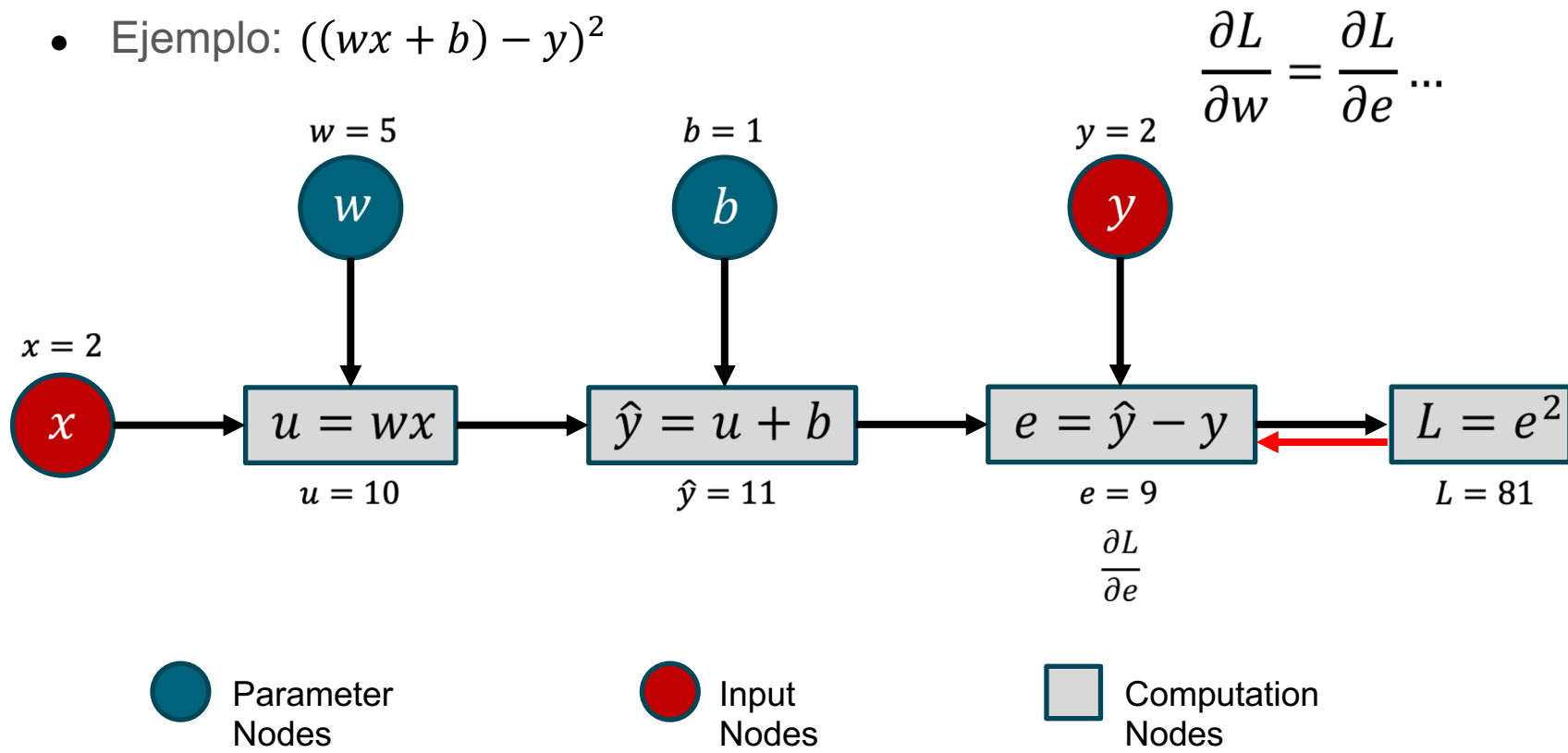
Regla de la cadena



$$\frac{df(g(x))}{dx} = \frac{df}{dg} \frac{dg}{dx}$$

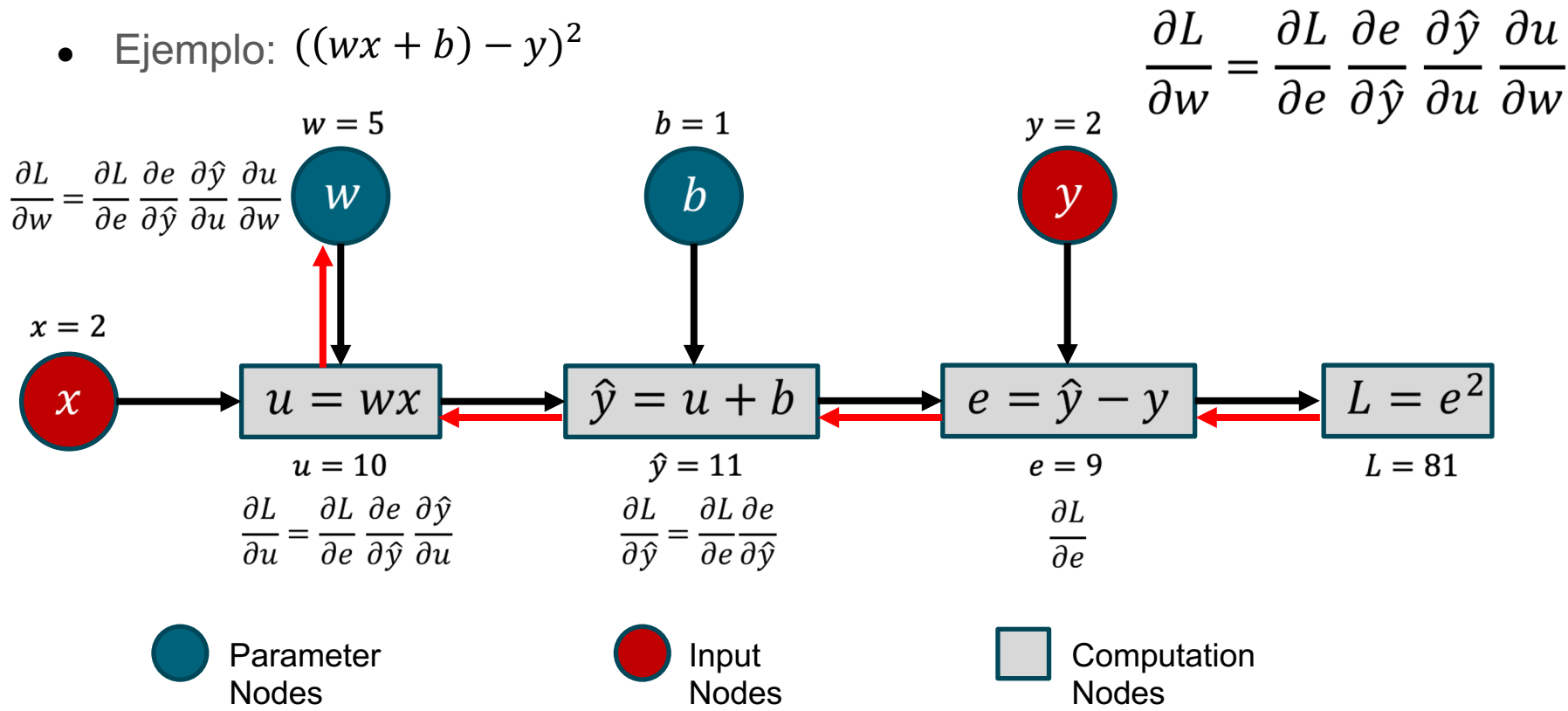
Grafos computacionales: Backward Pass

- Ejemplo: $((wx + b) - y)^2$



Grafos computacionales: Backward Pass

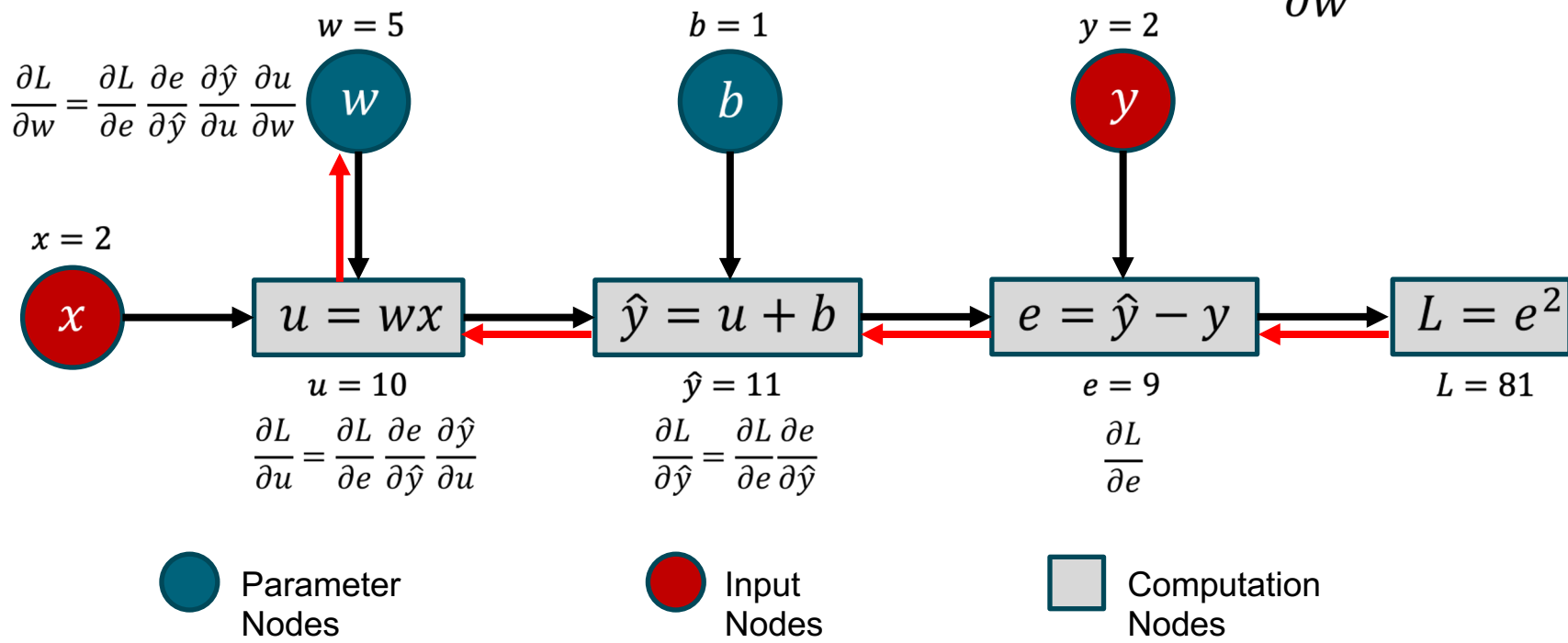
- Ejemplo: $((wx + b) - y)^2$



Grafos computacionales: Backward Pass

- Ejemplo: $((wx + b) - y)^2$

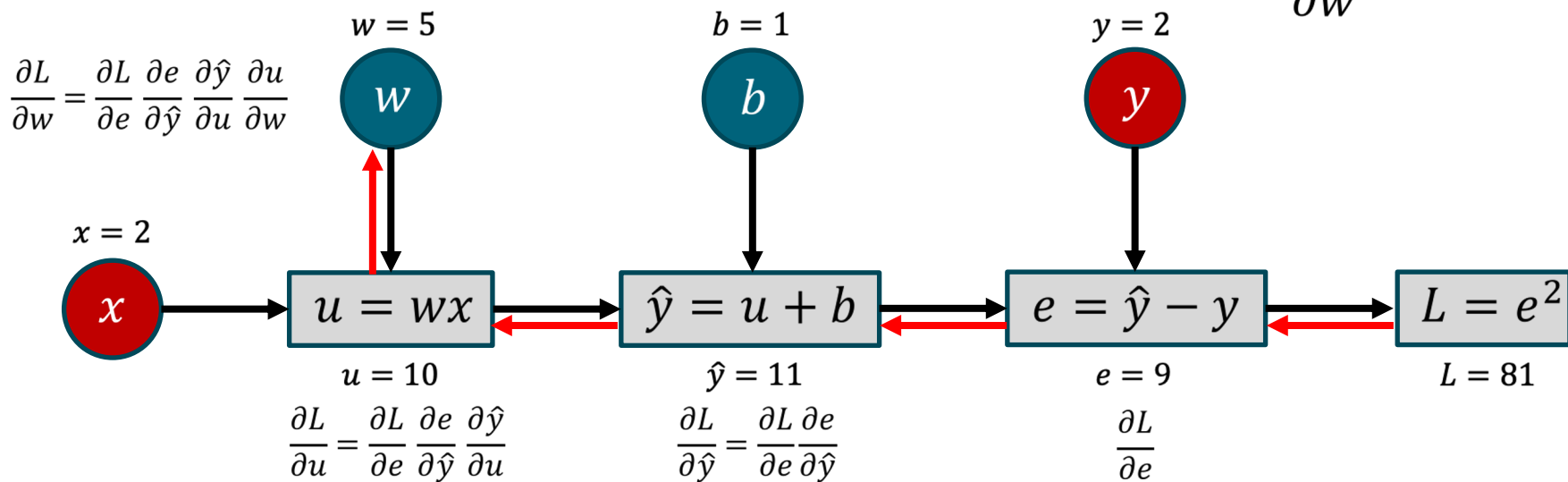
$$\frac{\partial L}{\partial w} = 2ex = 36$$



Grafos computacionales: Backward Pass

- Ejemplo: $((wx + b) - y)^2$

$$\frac{\partial L}{\partial w} = 2ex = 36$$



Parameter
Nodes



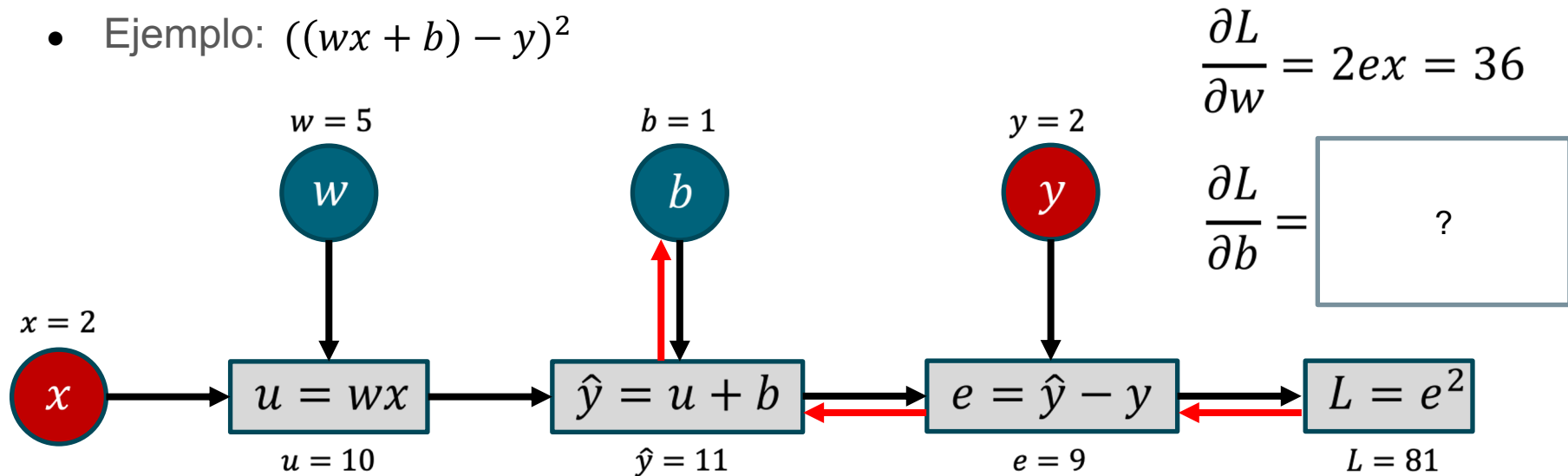
Input
Nodes



Computation
Nodes

Grafos computacionales: Backward Pass

- Ejemplo: $((wx + b) - y)^2$



Parameter
Nodes



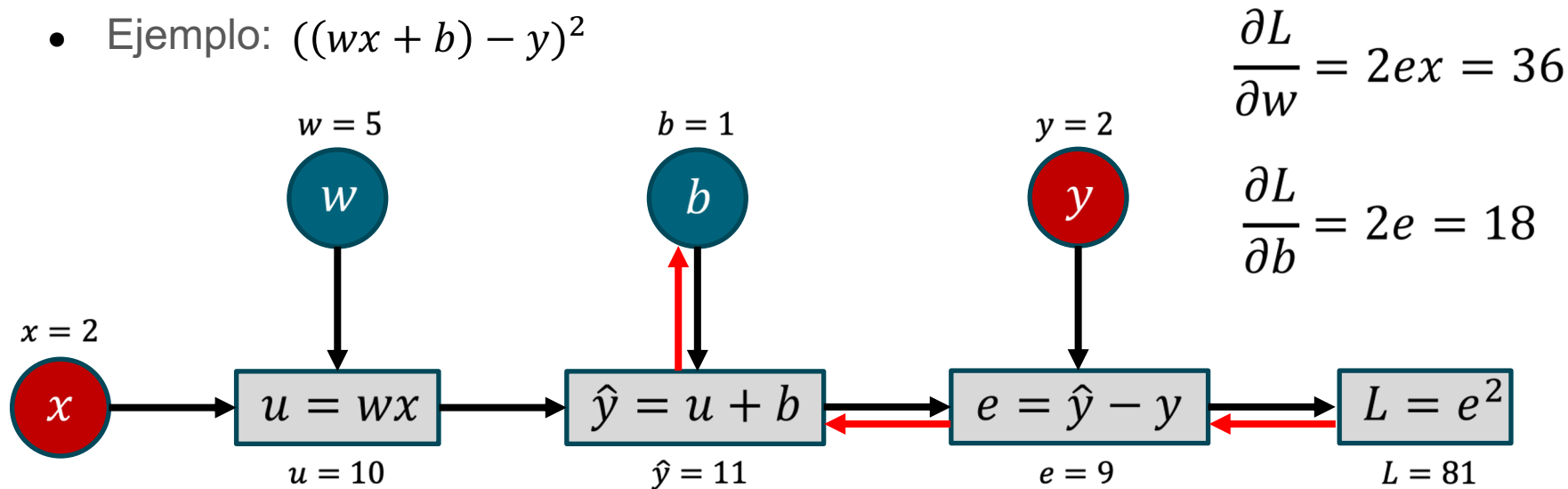
Input
Nodes



Computation
Nodes

Grafos computacionales: Backward Pass

- Ejemplo: $((wx + b) - y)^2$



Parameter
Nodes



Input
Nodes



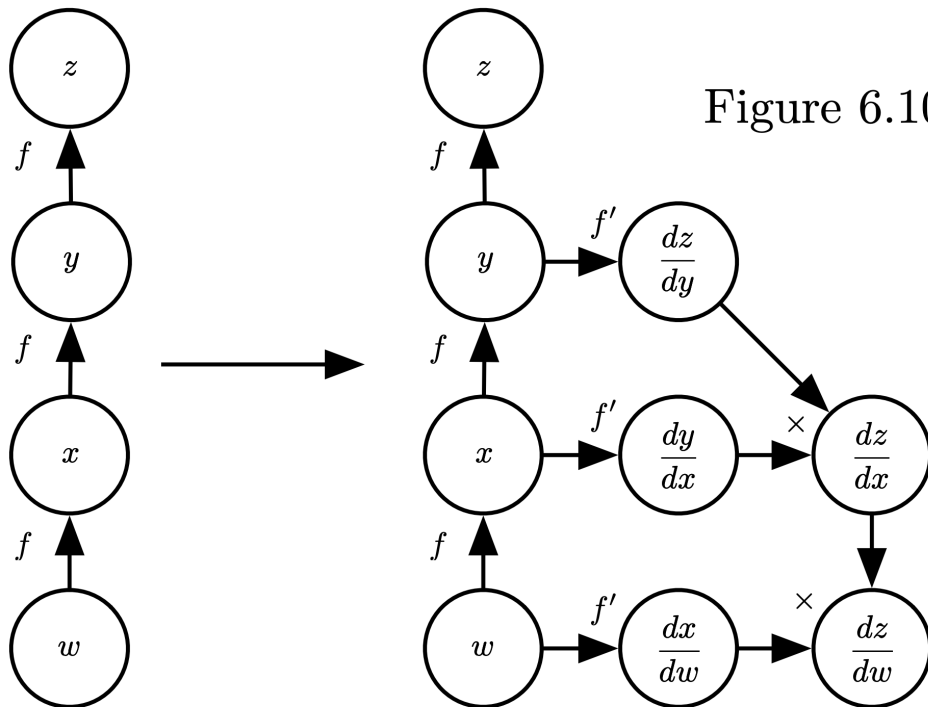
Computation
Nodes

Reutilizando calculos

- $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial u} \frac{\partial u}{\partial w}$
- $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b}$

¡Podemos reutilizar los cálculos ya hechos!

Reutilizando calculos



Ejemplos grafos computacionales

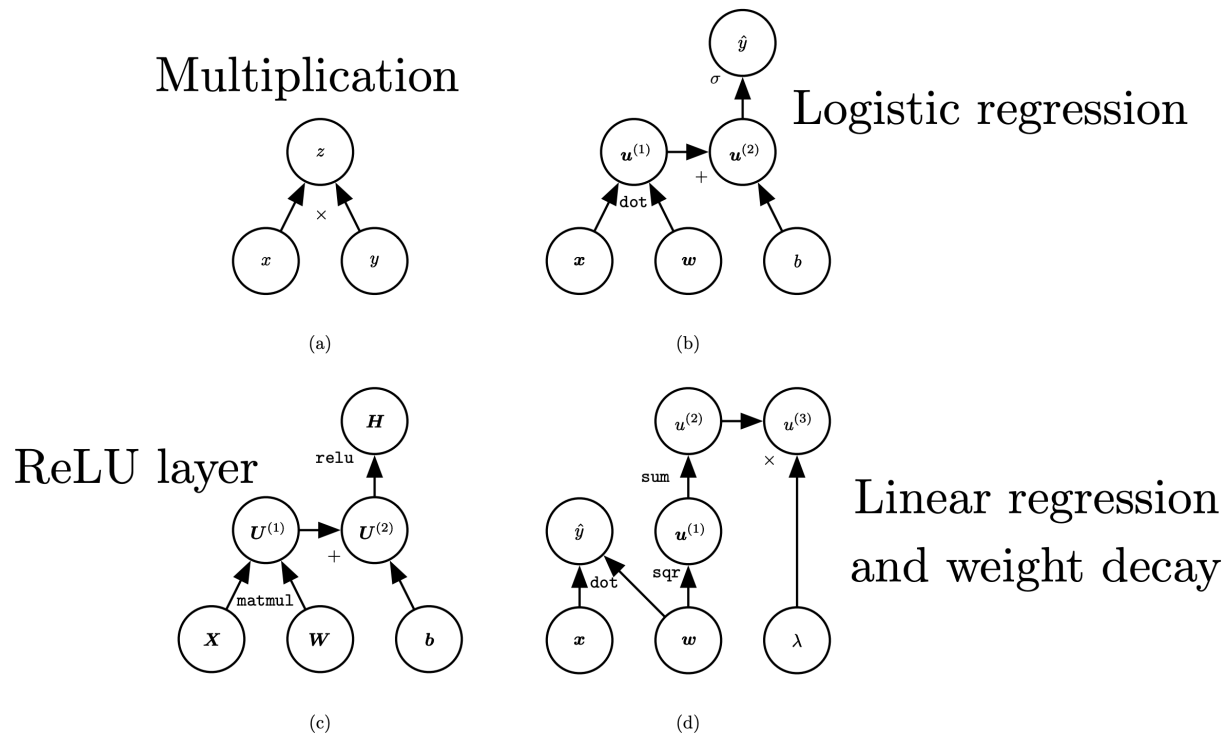
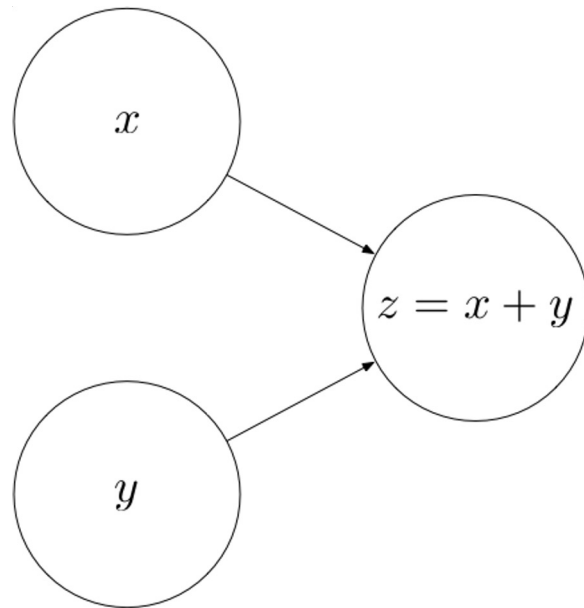


Figure 6.8

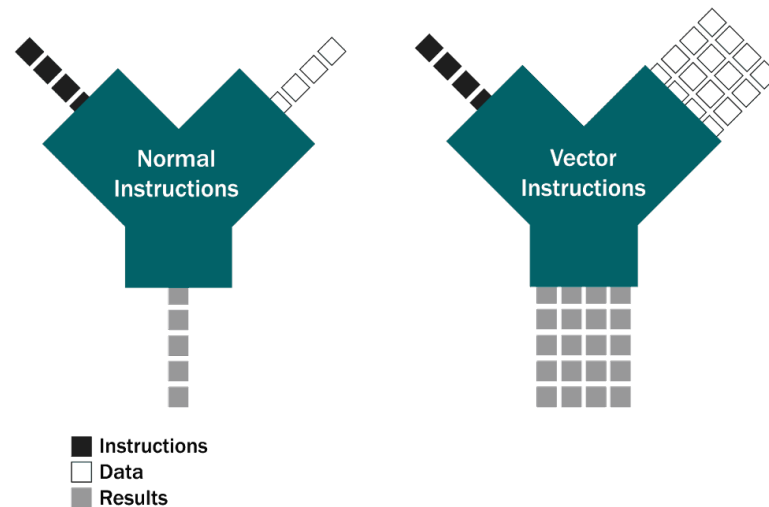
Grafos computacionales

- El ejemplo se extiende a redes enteras
- Los frameworks de redes neuronales construyen estos grafos (pero se pueden deshabilitar)
- Hay librerías de python que nos permiten visualizar dichos grafos.



Agenda

1. Introducción
2. Forward Propagation/Pass
3. Vector computation
4. Backward Propagation/Pass
 - a. Ejemplo
5. Grafos computacionales
6. **Conjunto de observaciones**
7. Resumen



Multiples observaciones en simultaneo

- Las redes neuronales procesan **multiples observaciones en simultaneo**
 - Ej: Predecir lluvia con humedad y temperatura
- La vectorizacion mejora la eficiencia y rapidez.

Humedad	Temperatura	Lluvia?
50%	70°F	NO
52%	73°F	NO
45%	70°F	NO
83%	96°F	YES
79%	97°F	NO
81%	90°F	NO
90%	91°F	YES
90%	89°F	YES

Computando múltiples observaciones juntas

- **Ejemplo:** 2 inputs y 3 unidades

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Computando múltiples observaciones juntas

- **Ejemplo:** 2 inputs y 3 unidades

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

$$\mathbf{z} = \begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Computando múltiples observaciones juntas

- **Ejemplo:** 2 inputs, 3 unidades y m observaciones

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad X = \underbrace{\begin{bmatrix} x_1^{(1)} & \dots & x_1^{(m)} \\ x_2^{(1)} & \dots & x_2^{(m)} \end{bmatrix}}_{\text{\#input} \times \text{\#observations (m)}} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Computando múltiples observaciones juntas

- **Ejemplo:** 2 inputs, m observaciones y 3 unidades

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad X = \begin{bmatrix} x_1^{(1)} & \dots & x_1^{(m)} \\ x_2^{(1)} & \dots & x_2^{(m)} \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

- **Forward pass**

$$Z = \begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & \dots & x_1^{(m)} \\ x_2^{(1)} & \dots & x_2^{(m)} \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Computando múltiples observaciones juntas

- **Ejemplo:** 2 inputs, m observaciones y 3 unidades

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad X = \begin{bmatrix} x_1^{(1)} & \dots & x_1^{(m)} \\ x_2^{(1)} & \dots & x_2^{(m)} \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

- **Forward pass**

$$Z = \begin{bmatrix} w_{11}x_1^{(1)} + w_{21}x_2^{(1)} & \dots & w_{11}x_1^{(m)} + w_{21}x_2^{(m)} \\ w_{12}x_1^{(1)} + w_{22}x_2^{(1)} & \dots & w_{12}x_1^{(m)} + w_{22}x_2^{(m)} \\ w_{13}x_1^{(1)} + w_{23}x_2^{(1)} & \dots & w_{13}x_1^{(m)} + w_{23}x_2^{(m)} \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Computando múltiples observaciones juntas

- **Ejemplo:** 2 inputs, m observaciones y 3 unidades

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad X = \begin{bmatrix} x_1^{(1)} & \dots & x_1^{(m)} \\ x_2^{(1)} & \dots & x_2^{(m)} \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

- **Forward pass**

$$Z = \begin{bmatrix} w_{11}x_1^{(1)} + w_{21}x_2^{(1)} + b_1 & \dots & w_{11}x_1^{(m)} + w_{21}x_2^{(m)} + b_1 \\ w_{12}x_1^{(1)} + w_{22}x_2^{(1)} + b_2 & \dots & w_{12}x_1^{(m)} + w_{22}x_2^{(m)} + b_2 \\ w_{13}x_1^{(1)} + w_{23}x_2^{(1)} + b_3 & \dots & w_{13}x_1^{(m)} + w_{23}x_2^{(m)} + b_3 \end{bmatrix}$$

Computando múltiples observaciones juntas

- **Ejemplo:** 2 inputs, m observaciones y 3 unidades

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad X = \begin{bmatrix} x_1^{(1)} & \dots & x_1^{(m)} \\ x_2^{(1)} & \dots & x_2^{(m)} \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

- **Forward pass**

$$\mathbf{a} = \begin{bmatrix} \varphi(w_{11}x_1^{(1)} + w_{21}x_2^{(1)} + b_1) & \dots & \varphi(w_{11}x_1^{(m)} + w_{21}x_2^{(m)} + b_1) \\ \varphi(w_{12}x_1^{(1)} + w_{22}x_2^{(1)} + b_2) & \dots & \varphi(w_{12}x_1^{(m)} + w_{22}x_2^{(m)} + b_2) \\ \varphi(w_{13}x_1^{(1)} + w_{23}x_2^{(1)} + b_3) & \dots & \varphi(w_{13}x_1^{(m)} + w_{23}x_2^{(m)} + b_3) \end{bmatrix}$$

Computando múltiples observaciones juntas

- **Ejemplo:** 2 inputs, m observaciones y 3 unidades

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \quad X = \begin{bmatrix} x_1^{(1)} & \dots & x_1^{(m)} \\ x_2^{(1)} & \dots & x_2^{(m)} \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

- **Forward pass**

$$Z = W^T X + \mathbf{b}$$

¿Por qué es importante?

- **Eficiencia:** Las operaciones con matrices se computan más rápido que iterando entre distintas observaciones.
- **Paralelismo:** Las operaciones de matrices se pueden paralelizar, especialmente con hardware como GPU.
- **Procesamiento Batch:** Las redes neuronales ofrecen entrenamiento en batches de datos en vez de por cada observación. La notación matricial soporta este proceso naturalmente.

Vectorization in NumPy

-
- Matrix multiplication vs. for loop:

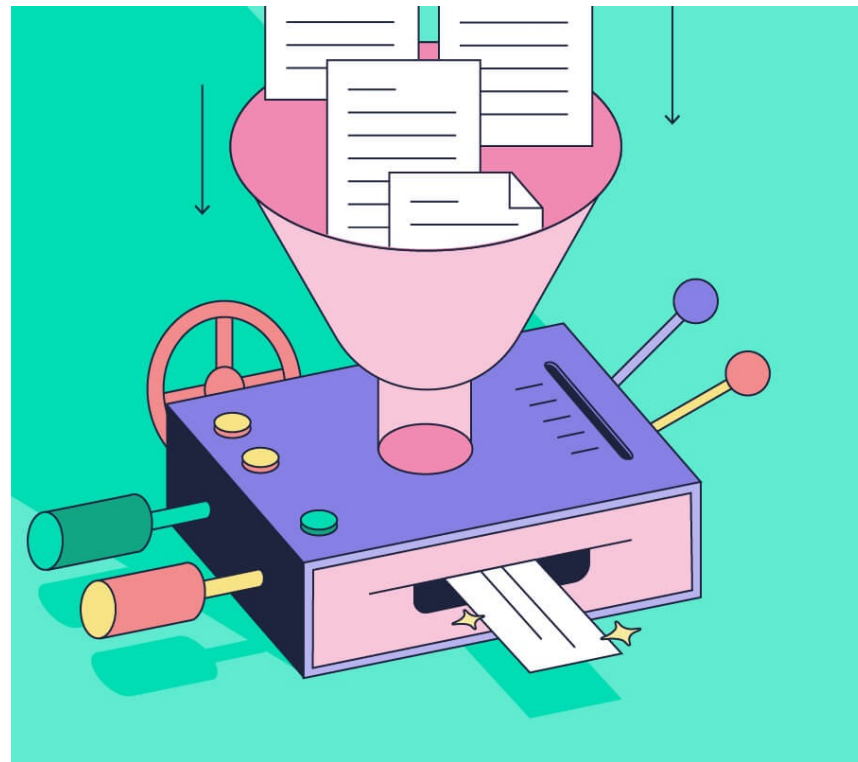
Ejercicio

- Comparar tiempo operaciones loop vs numpy

[Loop_vs_np_ex.ipynb](#)

Agenda

1. Introducción
2. Forward Propagation/Pass
3. Vector computation
4. Backward Propagation/Pass
 - a. Ejemplo
5. Grafos computacionales
6. Conjunto de observaciones
7. **Resumen**



Forward Pass

El proceso de pasar los inputs a través de la red para obtener un output

1. Calcular las operaciones lineales en el forward pass para cada capa.
2. Aplicar las funciones de activación.
3. Guardar los parámetros para utilizar en el backward propagation.

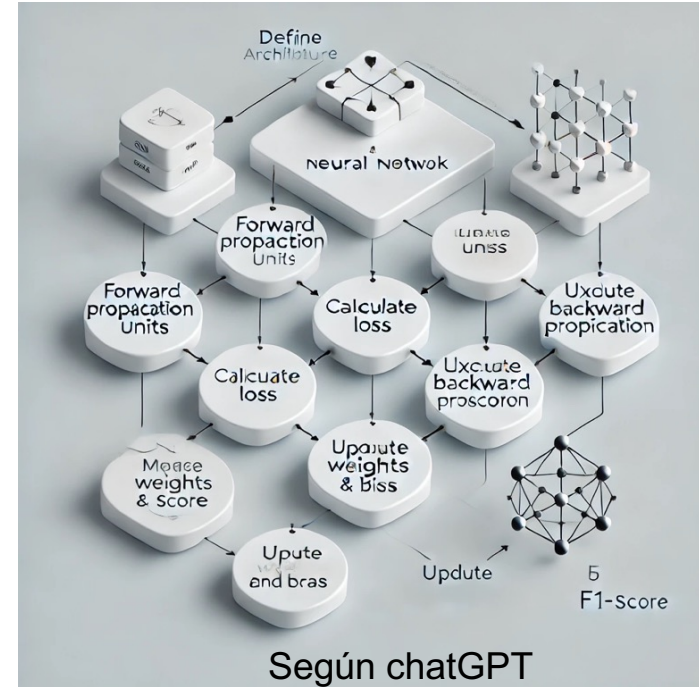
Backward Pass

El proceso de actualizar los pesos y bias utilizando el gradiente de la función de pérdida (loss).

1. Calcular la función de pérdida (loss).
2. Calcular los gradientes utilizando la regla de la cadena.
3. Aplicar el “learning rate” al gradiente.
4. Actualizar los pesos y bias.

Implementation Review

- Definir la arquitectura de la red neuronal: capas, unidades, funcion de activacion.
- Implementar el ciclo de entrenamiento
- Iterativamente ejecutar:
 - forward propagation
 - calcular la loss
 - ejecutar backward propagation
 - actualizar los pesos y bias.
- Medir la performance (e.g., accuracy, F1-score).
- Utilizar la red para realizar predicciones



Bibliografia

- [Deep Learning](#), Ian Goodfellow and Yoshua Bengio and Aaron Courville - Capítulo 6.5
- Neural Networks and Deep Learning. Charu C. Aggarwal 1.3